

队伍编号	MC25006634
题号	D

基于多阶段联合调度模型的短途运输货量预测与车辆优化研究

摘要

为响应《“十四五”现代物流发展规划》中提升物流效率与降低运营成本的核心目标，本文针对短途运输场景中货量预测偏差大、车辆调度效率低等问题，提出一种融合时序预测与动态优化的多阶段联合调度模型。通过构建混合预测框架、多目标优化模型及鲁棒性评估体系，系统解决货量精准预测、运输需求规划、标准容器适配和预测偏差影响分析等关键问题。

针对问题一：本文基于各线路历史 15 天的实际货量与当前未来一天的“预知”货量数据，考虑发运异常、订单取消与未下单因素等扰动因素，提出基于校正因子与自回归积分滑动平均（ARIMA）的混合预测模型，通过异常值剔除与历史分布比例拆解，实现线路级货量的 10 分钟颗粒度预测，得到了完整的未来 24 小时包裹量分布图，预测误差降低至 9.61%。

针对问题二：本文基于问题一的预测结果，按照车辆装载上限为 1000 件的约束，合理划分发运批次，并设计运输需求生成与车辆调度策略。模型引入自有车辆周转条件、串点合车规则与外部车辆替代机制，构建了以“车效最大化、成本最小化”为目标的调度优化方案。采用启发式规则结合贪心算法完成任务匹配与车辆调度，在规定的发运时间限制下，有效提升自有车辆的使用效率。

针对问题三：我们进一步引入标准容器的使用条件，考虑装卸时间缩短至 10 分钟、单装车载量下降至 800 件的变化，重新规划运输需求与车辆调度方案。通过建立布尔决策变量判定容器使用与否，在成本、效率间动态权衡，实现最优资源调度结构，并进一步降低系统运营总成本。

针对问题四：本文设计蒙特卡洛预测值模拟实验，研究问题一的货量预测结果出现偏差时，对问题三调度模型优化结果的影响。然后对优化模型进行鲁棒性分析。结果显示，模型在面对-10%到+10%级别的货量扰动时，仍能保持较优的车辆调度与成本优化结果，具备良好的稳定性与应用前景。

综上所述，本文构建的融合时序预测与动态优化的多阶段联合调度模型。具有良好的通用性与可扩展性，能够为物流网络末端的配送、车队资源管理及运输系统智能化提供理论支持与实践指导，具备一定推广价值。

关键词：短途运输；货量预测；多目标优化；标准容器；鲁棒性分析；调度优化

目录

一、 问题重述	1
(一) 问题背景	1
(二) 需要解决的问题	1
二、 问题分析	2
(一) 问题一	2
(二) 问题二	2
(三) 问题三	2
(四) 问题四	3
三、 模型假设	3
四、 符号说明	4
五、 模型建立与求解	5
(一) 问题一	5
1. 模型建立前的准备	5
2. 模型的建立	7
3. 模型的求解结果:	10
4. 模型预测效果检验	10
(二) 问题二	10
1. 模型的建立	10
2. 模型求解结果:	13
3. 模型结果分析	14
(三) 问题三	17
1. 模型建立	17
2. 模型求解结果	17
3. 模型结果分析	18
(四) 问题四	19
1. 模型建立	19
2. 模型结果分析	21
六、 模型评价与推广	23
(一) 模型的评价	23

1. 时间序列模型 ARIMA	23
2. 多目标优化模型	23
3. 贪心算法模型	23
4. 基于蒙特卡洛的预测值扰动模型	24
(二) 创新点	24
(三) 模型的改进	24
1. 时间序列模型 ARIMA	25
2. 多目标优化模型	25
3. 贪心算法模型	25
4. 基于蒙特卡洛的预测值扰动模型	25
(四) 模型推广	25
七、 参考文献	26
八、 附录	27

一、问题重述

（一）问题背景

《“十四五”现代物流发展规划》中提出：到 2025 年，基本建成供需适配、内外联通、安全高效、智慧绿色的现代物流体系。展望 2035 年，现代物流体系更加完善，具有国际竞争力的一流物流企业成长壮大，通达全球的物流服务网络更加健全，对区域协调发展和实体经济高质量发展的支撑引领更加有力，为基本实现社会主义现代化提供坚实保障。^[8]

在国民经济发展方面，物流业作为基础性的产业，它可以促进商品的周转、优化产业布局和提升居民消费品质。^[6]

党中央提出要建立以国内大循环为主体，国内国际双循环相互促进的新发展格局，在这一格局下，我国物流行业将承担起更大的责任，而作为物流网络中的最后一环节的短途运输，其具有运输距离短、资源可复用、时效性要求高的特点，对包裹履约时效和客户体验至关重要。而短途运输占用大量运力资源，现有物流技术手段无法充分解决短途运输中运货难、效率低、成本高、客户体验差等难题。因此，提升效率、降低成本，成为物流行业的首要解决难题。

（二）需要解决的问题

本文主要通过对附件中的数据进行分析、清洗，建立起合理的模型，对货量进行预测具体问题如下：

问题一：针对附件中给出数据，建立货量预测模型，对各条线路未来 1 天的货量进行预测，并将每条线路的总货量拆解到 10 分钟颗粒度，结果的时间范围为 12 月 15 日 14:00 至 12 月 16 日 14:00。需要将预测结果写入结果表 1 和表 2 中，并在论文中给出线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的预测结果

问题二：该问题要求根据问题 1 预测的货量结果，确定各条线路的运输需求，包括发运车辆数、预计发运时间及串点方案，确保发运时间不晚于发运节点。同时，需基于这些结果为每个需求分配承运车辆，优先提高自有车辆的周转率，最大化车辆均包裹量，并尽可能降低总成本。最终结果应写入结果表 3，包含线路编码、预计发运时间和承运车辆信息。串点方案需将多条线路编码合并显示，若无法使用自有车辆则标注为“外部”。此外，需在论文中详细给出线路编码为“场地 3-站点 83-0600”和“场地 3-”站点 83-1400”的调度结果。

问题三：该问题要求基于问题 1 的预测结果，在考虑使用标准容器的情况下重新确定运输需求。使用标准容器可将装车及卸车时长缩短至 10 分钟，但装载量降至 800 个包裹。需在问题 2 的优化目标基础上，重新确定各运输需求是否使用该容器，并将结果写入结果表 4。同时需在论文中给出指定线路编码的调度结果和是否使用标准容器。

问题四：该问题要求评估问题 1 中货量预测结果发生偏差时对项目 3 的调度模型优化结果的影响。

二、问题分析

（一）问题一

这个问题要求我们建立对未来一天每条线路货量的预测模型，并将其拆解到每 10 分钟颗粒度，在求解这个问题的重点在于我们需要对附件三中的“预知货量”中进行误差修正，对数据中存在的发运异常、未涵盖部分未下单的货量、已下单后取消的货量进行排除，以得到修正后的“预知货量”。然后将修正后的“预知货量”输入至随机森林、ARIMA 或 GRU 等机器学习模型实现货量拟合与修正，最后通过历史分布比例法将修正后的未来一天的预知货量拆解到 10 分钟颗粒度。

（二）问题二

问题二要求我们在第一问得出的货量预测结果的基础上，将其进一步地转化为运输需求。我们需要确定每条线路的发车次数、发运时间和车辆安排。题干中给出了以下几点约束条件：1.车辆单次装载量固定为 1000 件；2.每条线路运输任务的发运时间应早于发运节点；3.要求在车辆调度中优先使用自有车辆，并通过合理安排实现车辆的高效周转；4.最多允许三条同起点、目的地相近和可串点的线路合并装车发运，即对部分线路进行串点操作。本题是一个多目标优化问题，需要在约束条件限制下，根据每个子目标权重，最小化总成本、最大化自有车辆周转率和均包裹，从而最大化总目标函数。得到较优的车辆调度方案。

（三）问题三

问题三在逻辑上沿用了问题 2 的全部思路，也就是在问题 2 的基础上引入标准容器后的优化问

题。问题三引入了一种标准容器可以缩短装卸时间，从原来的 45 分钟减少到 10 分钟，但车辆的装载量会从 1000 降低到 800。同时，这种容器的数量是无限的，可以随意使用。问题三的优化目标与问题二相同。即最大化自有车辆周转率和车辆均包裹以及最小化总成本。

此外，还要输出每个运输需求是否使用该标准容器。这样一来，每次装车的临界量、发运时刻与车辆周转的时间窗都要重新进行计算。在运输需求生成和串点任务合并时，车辆满载门槛变为 800 件，因此需要更频繁的触发发运，而车辆完成运输并返回后的可用时间会更早，理论上会提高周转率，但在某些线路、某些时段也可能出现更多的空驶或半载的情况。

对于是否采用标准容器，我们的策略是对于每个运输需求，比较使用标准容器前后的总成本，如果使用标准容器后总成本降低，则我们选择使用该容器。最终我们同样得到一份结果表示，其中每条记录除线路编码还要发运时间、乘运车辆外，还要指明该需求是否使用了标准容器，并在论文中给出线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的调度结果。

（四）问题四

在问题一中，我们基于历史数据建立了货量预测模型，对未来 24 小时内每条线路的包裹量进行了预测。随后，在问题二和三中，我们利用预测结果制定了车辆调度计划、发运策略，并引入了标准容器以进一步提升运输效率。然而，在实际应用中，货量预测结果往往难以完全准确。由于多种不可控因素，如订单突发波动、数据采集误差、天气或节假日等，预测货量可能偏离真实值。因此，问题四核心目标是，在货量预测存在误差的前提下，系统如何评估已有调度模型所受影响的程度和调度模型的鲁棒性。所以我们采用了基于蒙特卡洛的预测值扰动模型来评估系统的鲁棒性。确保系统在不确定性条件下仍具有良好的响应性、适配性与最优性。这不仅是对现有调度策略的检验，也是对模型可推广性和真实场景应用能力的重要补充。

三、模型假设

- 1、假设外部承运商能够提供足够的车辆来满足自有车辆无法承运的运输需求，即不存在外部运力不足的情况
- 2、自有车辆在完成一个运输需求后，能够灵活地进行下一个运输需求的调度，只要满足前后的运输需求衔接条件（前一个需求的发运时间+装车时间+在途时间*2+卸车时间<后一个需求的发运时间），无需考虑额外的非业务性限制。

- 3、设附件 2 中各线路历史 15 天的实际包裹量数据能够代表各线路的典型货量波动模式，并且能够反映出各线路的货量趋势、季节性及随机波动等特点，可用于对未来的货量预测。
- 4、假设每天的两个发运节点（6 点和 14 点）是必须严格遵守的，所有线路的货物必须在此之前发出，且这两个发运节点对应的最早货量时间（前一天 21 点和当天 11 点）之后不再产生新的包裹量。
- 5、假设不考虑不可抗力因素（如自然灾害、交通事故等）对运输过程的影响，假设运输过程能够正常进行。
- 6、假设问题 4 中货量预测偏差服从正态分布，均值为零，方差由历史误差统计确定。

四、符号说明

符号	说明
i	表示第 i 条线路
D_{inr}	第 i 条路，第 n 天的实际包裹量
D_{inf}	第 i 条路，第 n 天的预测包裹量
α_{in}	第 i 条路，第 n 天的校正因子
b	货量时间段
c	车辆单次装载量为 1000
t_j	前一个需求的发运时间
G	总的线路数量
T_j	在途运输时间
Q_{ic}	第 i 条线路，第 c 个时间段的货量预测值
C_j	需求 j 由自有车辆执行的单位变动成本
$\hat{U}$	使用标准容器的装卸车时间之和
U	不使用标准容器的装卸车时间之和
W	车辆装载量

五、模型建立与求解

（一）问题一

1. 模型建立前的准备

（1）数据预处理

①我们将附件 2 中不同时间段的包裹量相加，得到了每日每十分钟每线路的累计包裹量，利用 python 画图工具得到了三维曲面图（图 1）。

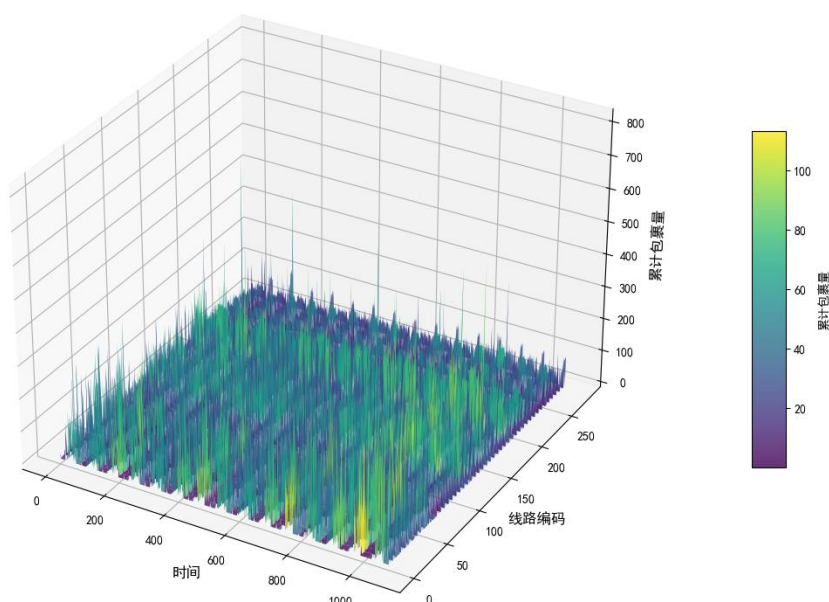


图 1

该三维曲面图通过对时间、线路、货量的三维映射，直观体现了货量高峰的时间段，为后续建立模型提供支撑。

②为了后续更好的计算结果，我们将附件 2 中以每十分钟颗粒度为步长的数据按照每一天、不同线路的数据进行相加，得到了每日各线路包裹量。为后面的均方根误差计算提供数据支撑。

（2）异常值剔除

3σ 原则是统计学和质量控制领域中一个重要的概念。在统计学范畴内，它基于正态分布的特性，具体而言，对于服从正态分布的数据，绝大部分数据（约 99.73%）会分布在均值 μ 减去 3 倍标准差 σ 到均值加上 3 倍标准差的范围内，即数值分布在 $(\mu - 3\sigma, \mu + 3\sigma)$ 这一区间内的概率为 0.9973。

正态分布的假设前提下，该原则能有效地识别数据中的异常情况和生产过程中的波动，为及时发现

并解决问题提供了有力的方法支持。

在建立模型前，我们先对附件 3 中的异常值进行剔除。首先，我们根据附件三中的包裹量得到了包裹量分布直方图（图 2）和归一化包裹量分布直方图（图 3），以包裹量为 X 轴，包裹量数值出现次数为 Y 轴，得到了关于包裹量的正态分布，接着，我们依据 3σ 原则，将分布在区间外的数值进行剔除，以得到正常值的附件三数据。

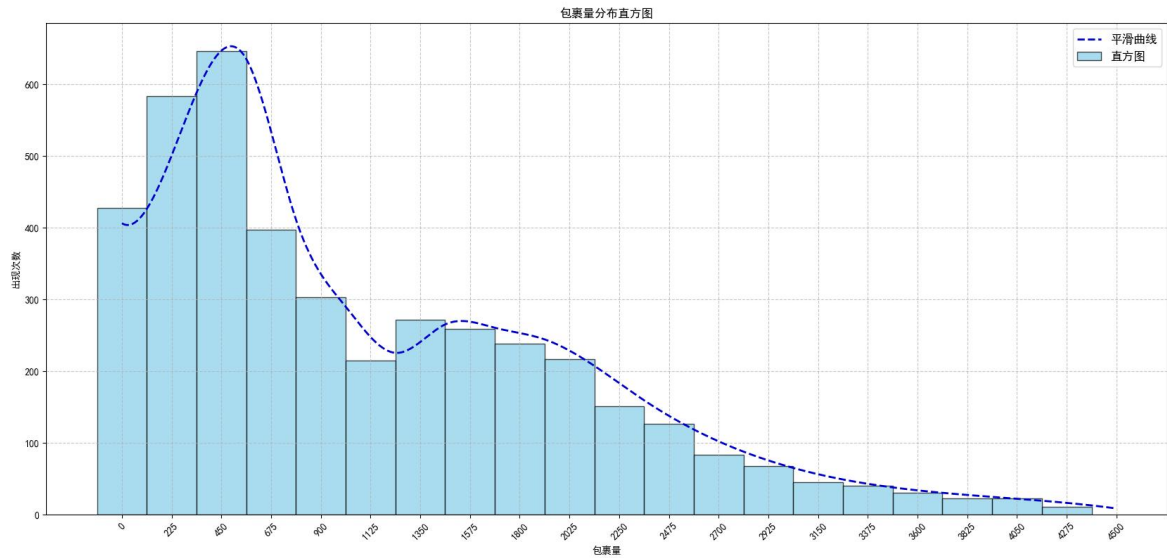


图 2

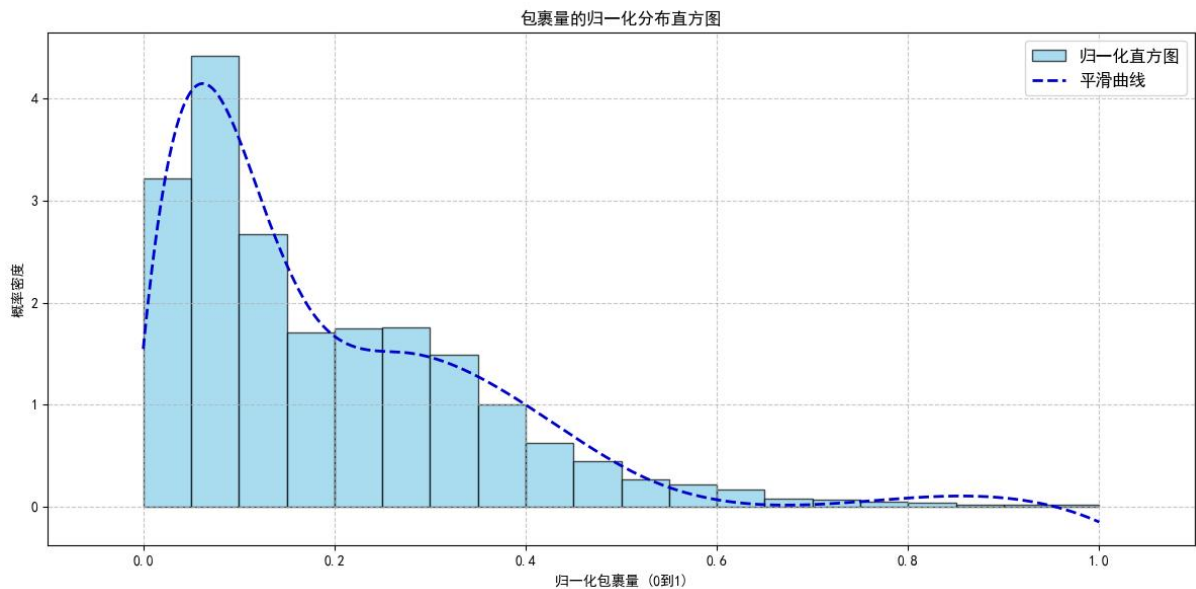


图 3

2. 模型的建立

(1) 校正因子 α 的引入

“预知”数据除了异常值外，还可能存在发运异常、未涵盖部分未下单的货量、已下单的货量可能会取消三个问题，为了解决这个问题，我们引入校正因子 α ，将剔除异常值后的附件三的“预知货量”与 α 因子进行相乘，以得到修正后的“预知货量”。

α 因子为附件二中 1-15 天的“实际货量”与附件三中 1-15 天“预知货量”的比值，公示如下：

$$\alpha_{in} = \frac{1}{n} \sum_{j=1}^n \frac{D_{inr}}{D_{inf}} \quad (1.1)$$

(2) 修正预测量模型的构建

将附件三中的“预知货量”与 α 因子相乘得到修正预测量模型，其中 Q_{inn} 表示第 i 条线路，第 n 天的修正后的“预知”包裹量， Q_{inf} 表示第 i 条线路，第 n 天的修正前的“预知货量”，公式如下：

$$Q_{inn} = Q_{inf} \cdot \alpha_{in} \quad (1.2)$$

(3) 自回归积分滑动平均模型 (ARIMA)

ARIMA 模型是 ARMA 模型的扩展，适用于非平稳时间序列，对多种类型的时间序列都具有较强的适用性。它主要由三部分构成，分别为自回归模型 (AR)、差分过程 (I) 和移动平均模型 (MA)。ARIMA 模型通过差分操作使非平稳时间序列转化为平稳时间序列，再对平稳时间序列进行 ARMA 模型拟合。ARIMA 模型的三个主要参数分别是：自回归项数 (p)、差分次数 (d)、移动平均项数 (q)，其核心公式如下：

$$X_t = c + \Phi_1 X_{t-1} + \Phi_2 X_{t-2} + \cdots + \Phi_p X_{t-p} + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q} + \varepsilon_t \quad (1.3)$$

公式中 X_t 表示第 t 天，某线路的货量， c 表示模型的常数项，也叫做截距项； $\Phi_1, \Phi_2, \dots, \Phi_p$ 表示 AR 模型的参数，表示过去值对当前值的影响系数； $\theta_1, \theta_2, \dots, \theta_q$ 表示模型 MA 的参数，表示过去 q 个误差项的影响系数。 ε_t 表示时间点的误差项，也被称为白噪声项。即均值为 0、方差为 σ^2 独立同分布随机变量。

我们将修正后的“预知货量”输入 ARIMA 模型实现货量拟合与修正，得出所要求的第 16 天“预知货量”。实际货量与预测货量的拟合图 4 所示：

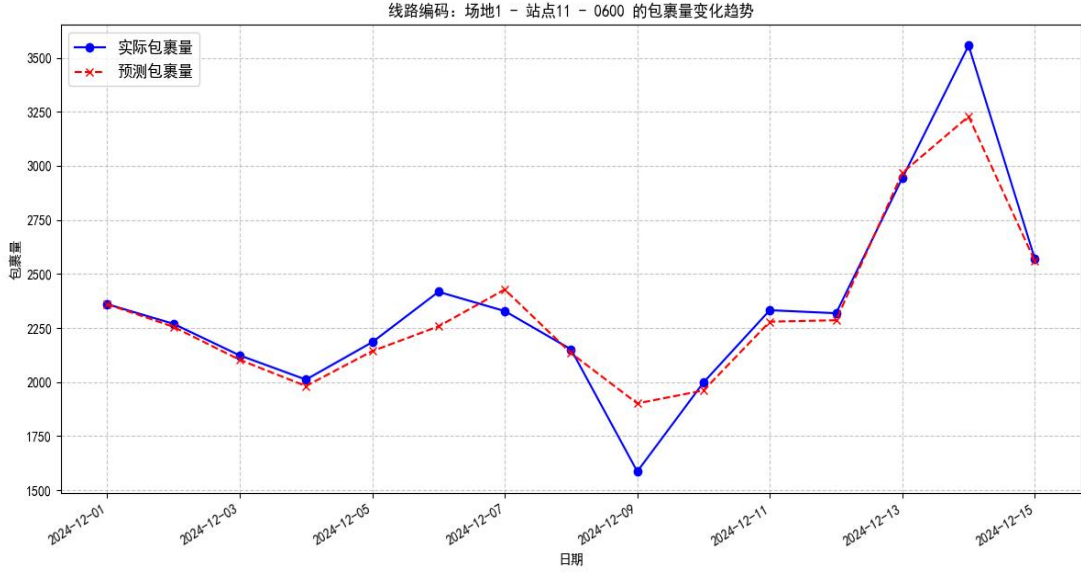


图 4

(4) 预测数据颗粒度拆解

①历史分布比例法概述

在得到未来一天的预测数据后，需要将每条线路的总货量拆解为每 10 分钟颗粒度的货量数据，我们将附件二中不同线路的 15 天内以 10 分钟颗粒度为步长的在某一时刻的历史货量相加，并将其除以这条线路 15 天的历史总货量以此得到某一时刻货量的占比情况。我们将其称为“历史分布比例法”并且通过 python 画图工具得到了路线场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的“包裹量随时间变化图”，具体见图 5 与图 6

②历史分布比例法核心公式

$$P_{ic} = \frac{K_{ic}}{\sum_{n=1}^{15} m_{in}} \quad (1.4)$$

P_{ic} : 第 i 条线路, 第 b 个时间段的货量占比

K_{ic} : 第 i 条线路, 第 b 个时间段的历史货量总和

m_{in} : 第 i 条线路, 第 n 天的货量总和

$\sum_{n=1}^{15} m_{in}$: 第 i 条线路, 第 1 到第 15 天的货量总和

$$Q_{ic} = Q_{16} \cdot P_{ic} \quad (1.5)$$

Q_{ic} : 第 i 条线路, 第 c 个时间段的货量预测值

Q_{16} : 第 16 天的货量总和预测值

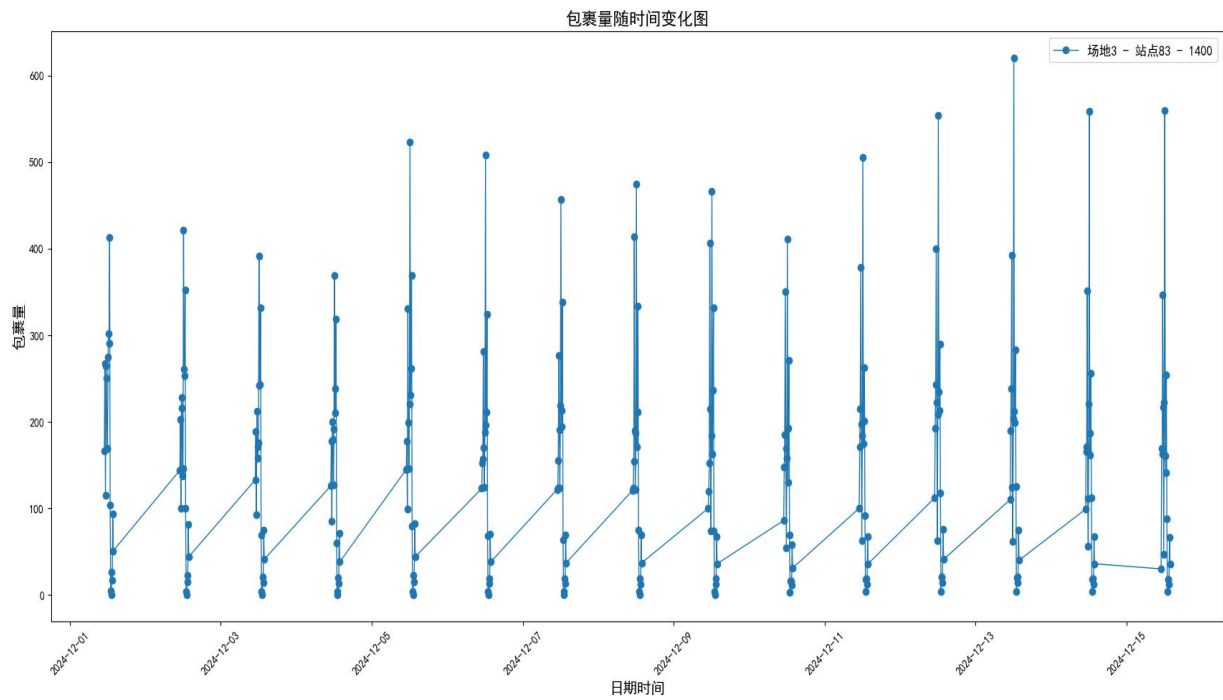


图 5

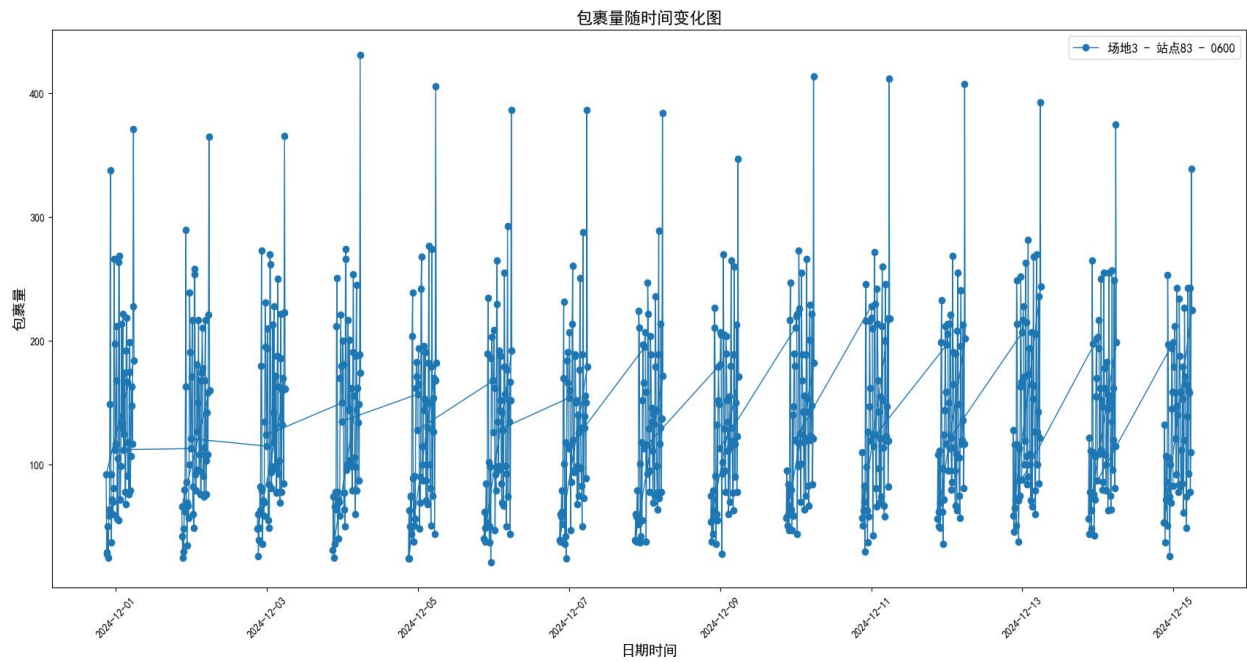


图 6

从图 5 和图 6 可以看出，线路每天货量呈现周期性变化，每天 6:00 和 14 点是线路货量的高峰，同时也是车辆的发运高峰。

3. 模型的求解结果：

依据上述模型，我们求解出了未来一天的预测货量。表 1 是线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的预测结果。

表 1

线路编码	日期	货量
场地 3 - 站点 83 - 0600	2024 年 12 月 16 日	7038
场地 3 - 站点 83 - 1400	2024 年 12 月 16 日	2515

4. 模型预测效果检验

我们将附件 3 修正后得到的“预知货量”（12.1-12.15）与附件 2（12.1-12.15 天）的“实际货量”利用 python 作均方根误差检验，得到结果为：179.38。我们计算相对均方根误差约为：9.61%，因此我们认为模型预测效果较好。

（二）问题二

1. 模型的建立

问题二是根据未来一天各线路的货量预测，确定每条线路承运的自有车辆和外部车辆发运时间和串联方案。因此，我们采用了多目标混合优化模型，以设定各目标期望值，并最小化偏离这些目标的程度，以实现自有车辆的周转率和所有车辆均包裹率最大化，以及总成本最小化的目标。首先对每条线路按时间累积货量，每当天累计货量达到 1000 时，生成一个货运需求，剩余货量继续累加，直到接发运节点时间。接着再串点情况下合并多个线路货量，以减少总体需求的数量，最后使用贪心算法解决车辆调度问题，尽可能为每一个需求分配最早所自用车辆。如果无自用车辆，则用外部车量，这三个步骤相互衔接，形成一个完整的调度方案生成过程。

（1）决策变量的确定

$$x_{jk} = \begin{cases} 1 \\ 0 \end{cases} \quad : \text{需求 } j \text{ 由自有车辆承运时为 1, 否则为 0}$$
$$y_j = \begin{cases} 1 \\ 0 \end{cases} \quad : \text{需求 } j \text{ 由外部车辆承运时为 1, 否则为 0}$$

$$z_{jg} = \begin{cases} 1 \\ 0 \end{cases} \quad : \text{需求 } j \text{ 属于串点任务组 } g \text{ 时为 1, 否则为 0}$$

t_j : 需求 j 的实际发运时间

(2) 目标函数建立

①最大化自有车辆周转率函数:

$$MaxZ_1 = \frac{\sum_{jk} X_{jk}}{V_d} \quad (2.1)$$

此函数为全天自有车辆承运需求数与自有车辆数之比

V_d : 某个车队的自有车辆数, $d \in \{1, 2, 3, 4, 5\}$

②最大化均包裹数函数:

$$MaxZ_2 = \frac{\sum_j Q_j}{\sum_v u_v + \sum_j y_j} \quad (2.2)$$

此函数为全天车队承运总包裹量与使用的自有车辆数和外部承运商车辆数之和的比

注: 如果均包裹量越大, 说明自有车辆使用率越高, 减少了空载和半载的损耗.

Q_j : 车队承运总包裹数

$\sum_v u_v$: 自有车辆使用数

$\sum_j y_j$: 外部承运商车辆数

③最小化总成本函数:

$$MinZ_3 = \sum_v u_v \cdot S + \sum_v \sum_{j \in v} X_{jk} \cdot C_j + \sum_j y_j \cdot e_j \quad (2.3)$$

e_j : 外部承运商承运的单位成本

C_j : 需求 j 由自有车辆执行的单位变动成本

S : 固定成本 100

④总目标函数:

$$MaxZ = \omega_1 \cdot Z_1 + \omega_2 \cdot Z_2 - \omega_3 \cdot Z_3 \quad (2.4)$$

$\omega_1, \omega_2, \omega_3$ 为指标权重, 用户可根据对目标关注度大小来调整这些指标的权重, 在本题中我

们更专注于总成本的最小化，因此指标权重设置为 $\omega_1 = 0.5$ ， $\omega_2 = 0.25$ ， $\omega_3 = 0.25$

(3) 约束条件

①需求覆盖约束：

$$\sum_k x_{jk} + y_j = 1, \quad \forall j \quad (2.5)$$

注：每个运输要求都会被满足，必须由自有车辆或者外部车辆承运

②自有车辆 k 的任务时间约束：

$$t_j \geq t_{j'} + 2 \cdot T_{j'} + U \quad (2.6)$$

$t_{j'}$ ：前一个需求的发运时间

$T_{j'}$ ：在途运输时间

j' ：车辆 k 在承运需求 j 之前的最后一个任务

U ：装车时间与卸车时间之和

③发运时间约束：

$$t_j \leq \psi_j \quad (2.7)$$

注： ψ_j 为 6:00 或 14:00 且发运时间不得晚于发运节点时间

④串点任务约束：

$$\sum_{j \in g} z_{jg} \leq 3, \quad \forall g \quad (2.8)$$

$$z_{jg} = \begin{cases} 1 \\ 0 \end{cases} \quad (2.9)$$

注： $z_{jg} = 1$ 表示需求 j 属于串点组 g ，即该需求与其他需求合并发运

g ：用与用于标识串点任务合并组的索引，明确哪些需求被合并为一个任务

⑤货量限制约束：

$$\sum_j z_{jg} \cdot Q_j \leq 1000, \quad \forall g \quad (2.10)$$

注：合并后总货量不超过车辆单次可装载量

⑥货量满载约束：

$$Q_j = \begin{cases} 1000 & \text{非串点任务} \\ \sum_{j \in g} R_j & \text{串点任务} \end{cases} \quad (2.11)$$

R_j :需求 j 的剩余货量

(4) 调度策略建立

①任务分配策略：

根据每条线路的货量预测值，按照时间累积货量，每当天累计货量达到一定阈值时，生成一个货运需求。这个阈值的设定与车辆的装载量相关，例如当车辆装载量固定为 1000 件时，当货量累计达到 1000 件左右就生成一个货运需求。

对于剩余的货量继续累加，直到接近发运节点时间，以确保在规定的时间内完成所有货运任务。

②车辆调度策略：

自有车辆优先调度：首先为每个货运需求分配自有车辆。按照车辆的可用时间（考虑装车、在途运输、卸车等时间因素）和当前所处位置，为每个需求匹配最适合的自有车辆。如果自有车辆能够满足需求，则进行分配；否则，再考虑使用外部车辆。

车辆任务衔接优化：确保自有车辆在完成一个运输任务后，能够及时进行下一个任务。这需要考虑车辆的周转时间，包括从上一个任务的卸车地点到下一个任务的装车地点的行驶时间，以及装车和卸车所需的时间等。

③串点任务策略：

合并条件判断：对于同一起点且目的地相近的线路，判断是否可以串点合并。主要考虑因素包括线路的货量总和是否在车辆装载量的范围内、各线路的发运时间是否能够协调等。

合并方案生成：将符合条件的多条线路合并为一个串点任务，优化运输路径和资源利用。通过合理的线路组合，减少所需车辆的数量，提高运输效率。

2. 模型求解结果：

我们依据上述建立的多目标混合优化模型实现了自有车辆的周转率和车辆均包裹率最大化，以及总成本最小化，确定了每条线路的需求。接着利用贪心算法求解出了最优调度结果，并将其填入表格三中。为线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的调度结果。

表 2

线路编码	日期	预计发运时间	发运车辆
场地 3-站点 83-0600	2024/12/15	22:00:00	外部车辆
场地 3-站点 83-0600	2024/12/16	00:00:00	外部车辆
场地 3-站点 83-0600	2024/12/16	01:00:00	外部车辆
场地 3-站点 83-0600	2024/12/16	03:00:00	外部车辆
场地 3-站点 83-0600	2024/12/16	05:00:00	外部车辆
场地 3-站点 831400	2024/12/16	12:00:00	外部车辆

3. 模型结果分析

(1) 关键指标达成

① 周转率最大化

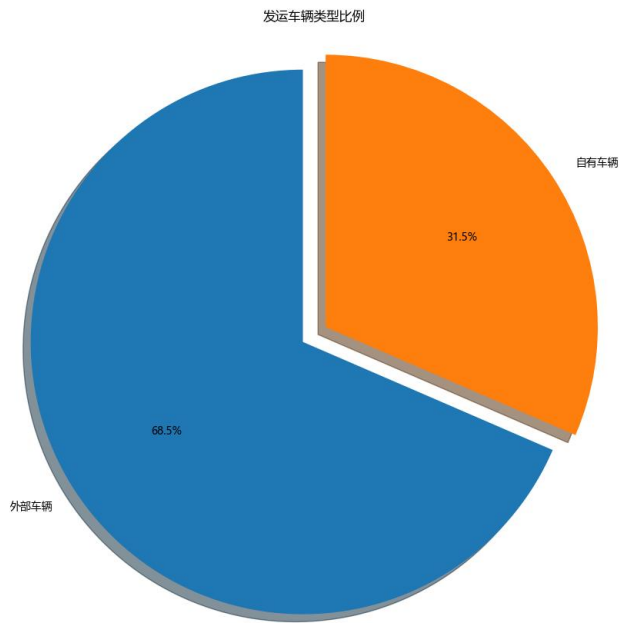


图 7

从图 7 可以看出，自有车辆在总体调度方案中的使用比例 31.5%，外部车辆占 68.5%，说明外部车辆在总体调度方案中是主要的运力来源。反映了模型结果达到了较高的自有车辆周转率。

② 均包裹最大化

由公式 2.2 得车辆均包裹为 865，车辆满载装载量约为 1000，均包裹接近满载量，车辆装载率约为 86.7%，说明车辆利用效率高。

③总成本最小化

由公式 2.3 结果可知，固定成本占总成本占比为 30%，变动成本占总成本占比为 15%。外部成本占总成本 55%，由此说明，外部车辆使用率高，此时总成本最小。

(2) 结果可视化

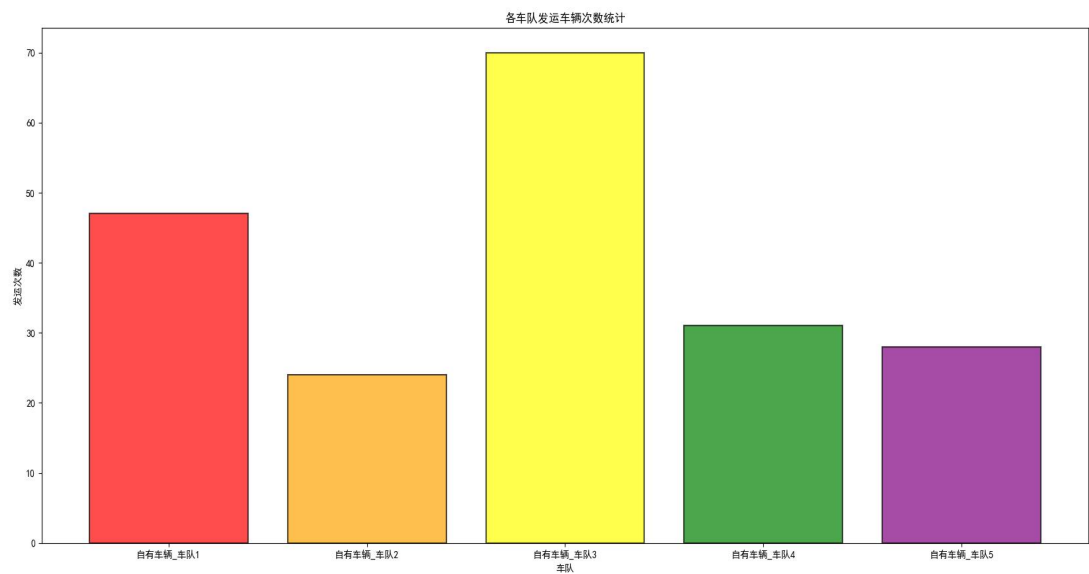


图 8

图 8 为各车队发运车辆次数统计图，该条形图显示了各车队自有车辆的使用频次。车队 1 至车队 5 的使用次数分别为 94 车队 3 次、81 次、124 次、65 次和 43 次。使用频率最高，说明该车队负责的路线需求最为集中，车队 5 使用频率最低，其负责的线路可能分布较为分散或总体需求较低。

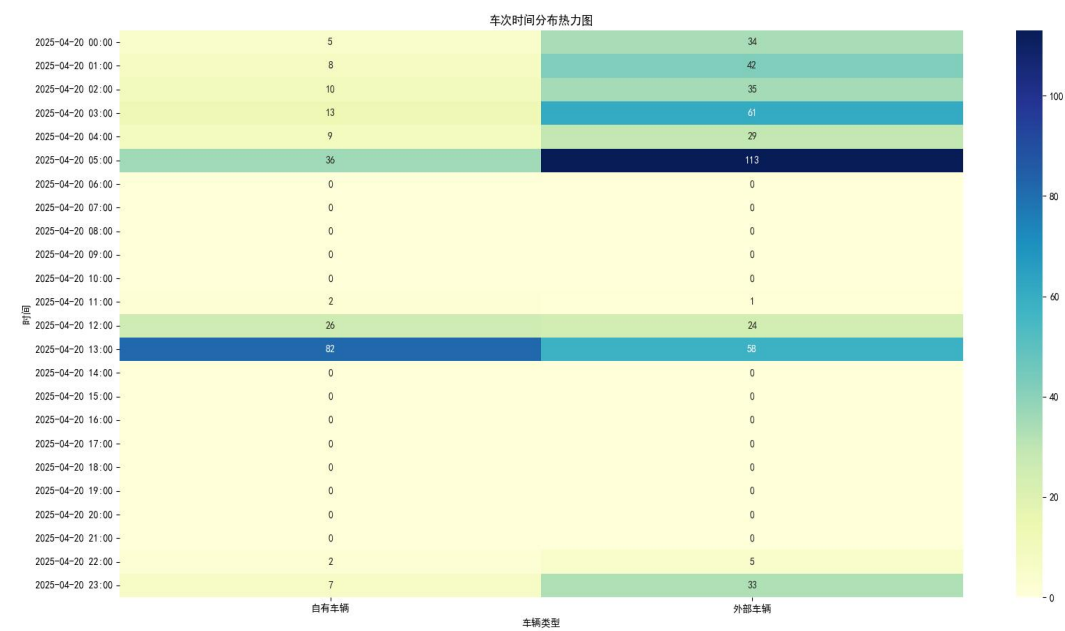


图 9

图 9 为车次时间分布热力图，该热力图直观展示了 24 小时内不同类型车辆的发运分布情况。由图可知，6 点，14 点是发运的高峰，这与两个关键发运节点时间一致。自有车发运时间主要集中在 0~6 点、9~14 点两个时间段，而外部车辆更多的补充了其他时段的运力需求。该模型反映了对车辆、时间、资源的合理分配。

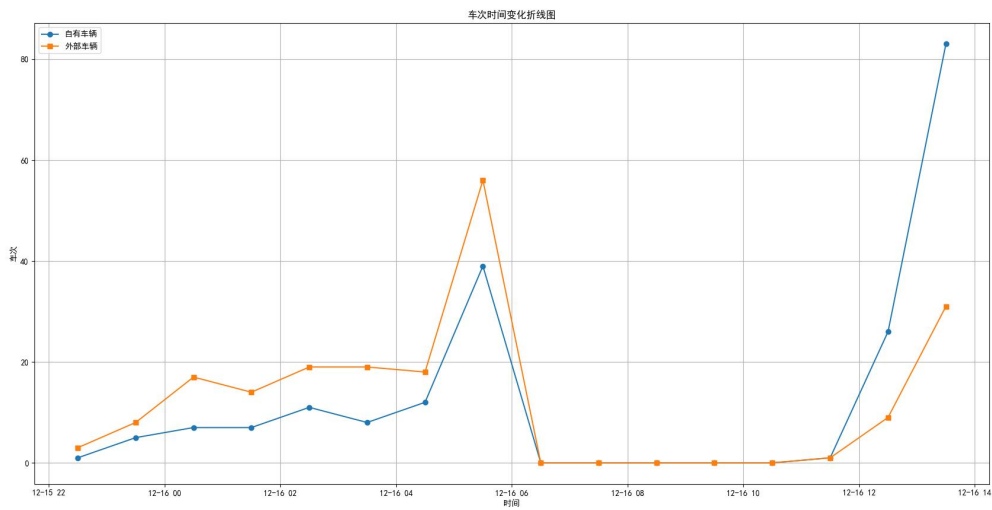


图 10

图 10 为车次时间变化折线图，它描绘了不同时间点车辆需求，蓝线代表自有车辆，黄线代表外部车辆。图中可见多个需求高峰，特别是在 6 点与 14 点附近，自有车的需求达到最高，自有车辆和外部车辆需求变化相近，当自有车辆需求增加时，外部车辆的需求也增加。

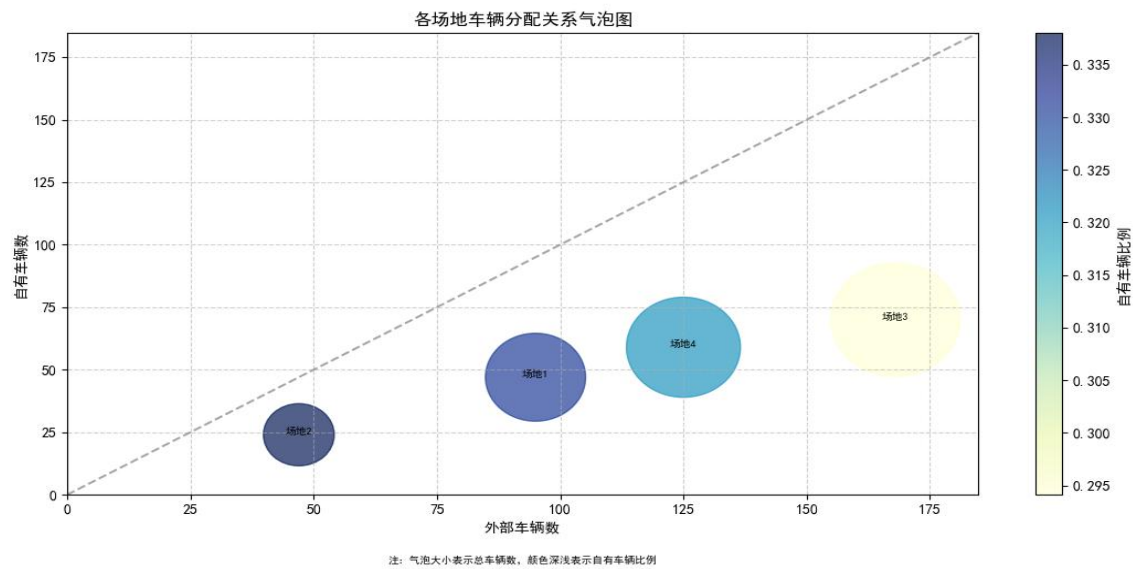


图 11

图 11 展示了各场地自有车与外部车的分配关系。其中气泡大小表示总车辆数，颜色深浅表示自由车的比例大小。从图中可以看出，场地 3 的气泡最大，表明其运输需求最高。场地 3 位于对角线下方，反映其外部车辆使用比率高于自由车辆。

（三）问题三

1. 模型建立

（1）在问题二原有模型的决策变量的基础上，增加容量约束和更改自有车辆任务时间约束。

（2）目标函数建立

①标准容器决策变量

$$p_j = \begin{cases} 1, & \text{若需求}j\text{使用标准容器} \\ 0, & \text{若使用常规容器} \end{cases} \quad (3.1)$$

②总目标函数

$$MaxZ = \omega_1 \cdot Z_1 + \omega_2 \cdot Z_2 - \omega_3 \cdot Z_3 \quad (3.2)$$

注：指标权重设置为 $\omega_1 = 0.5$ ， $\omega_2 = 0.25$ ， $\omega_3 = 0.25$

（3）约束条件

在问题二的约束条件上，增加容量约束和更改自有车辆任务时间约束

①容量约束

$$\sum_{j \in I} Q_j x_{jv} \leq \sum_{j \in I} [(1 - z_j) \cdot 1000 + z_j \cdot 800] \cdot x_{jv}, \quad \forall v \in V \quad (3.3)$$

②车辆任务时间约束

$$\forall v, \forall j, k, s, t, x_{jv} = x_{kv} = 1, j \neq k :$$

$$t_i + [(1 - p_j) \cdot 45 + p_j \cdot 10] + 2t_j + [(1 - p_j) \cdot 45 + p_j \cdot 10] \leq t_k \quad (3.4)$$

（4）调度策略建立

① 若该线路为自有车辆运输，优先考虑使用标准容器若线路货量刚好适配容器容量(如<800)，优先使用。

②若该线路需在高峰时段(如清晨 06:00 前)发运，使用容器以提高效率。

③若多个任务串点为一组，统一使用与否，避免调度冲突。

2. 模型求解结果

在利用多目标混合优化模型确定预测货量和发运节点数据，我们枚举出所偶有线路的货量 Q 和发运时间 T 的容器使用决策，并且使用贪心算法判定容器使用方案。最后将结果填入了结果表 4，线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的调度结果如表 3 所示。

表 3

线路	日期	预计发运时间	发运车辆	使用标准容器
场地 3 - 站点 83 - 0600	2024/12/15	22:00:00	外部车辆	TRUE
场地 3 - 站点 83 - 0600	2024/12/16	00:00:00	外部车辆	TRUE
场地 3 - 站点 83 - 0600	2024/12/16	01:00:00	外部车辆	TRUE
场地 3 - 站点 83 - 0600	2024/12/16	03:00:00	外部车辆	TRUE
场地 3 - 站点 83 - 0600	2024/12/16	05:00:00	外部车辆	TRUE
场地 3 - 站点 83 - 1400 - 场地 1 - 站点 35 - 1400	2024/12/16	11:00:00	外部车辆	TRUE
场地 3 - 站点 83 - 1400	2024/12/16	12:00:00	外部车辆	TRUE

3. 模型结果分析

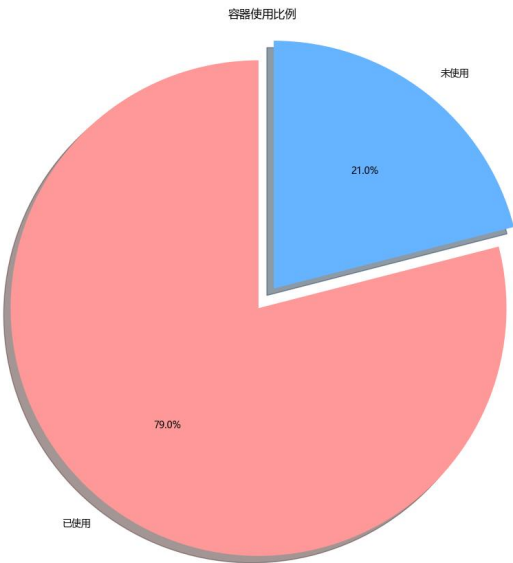


图 12

图 12 为标准容器使用比例图，展示了标准容器的使用情况，可以看出标准容器在这天使用比例为 79%，未使用比例为 21%，说明当天参与运输的车辆中，绝大多数使用了标准容器。可见标准容器引入可降低货物运输的总成本。

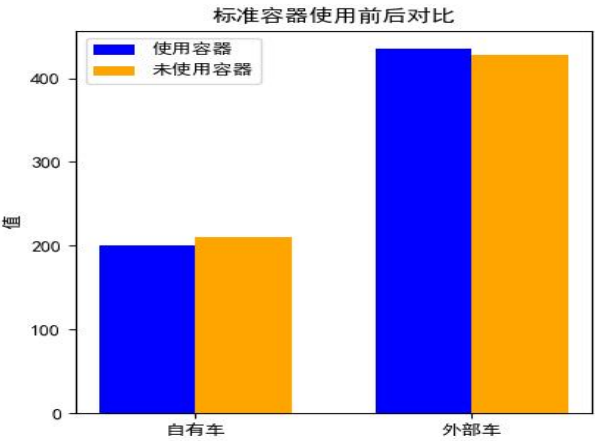


图 13

图 13 展示了标准容器使用前与后自有车与外部车使用数量的对比，由图可知使用标准容器后，自有车辆使用数量减少，外部车辆使用数量增加。

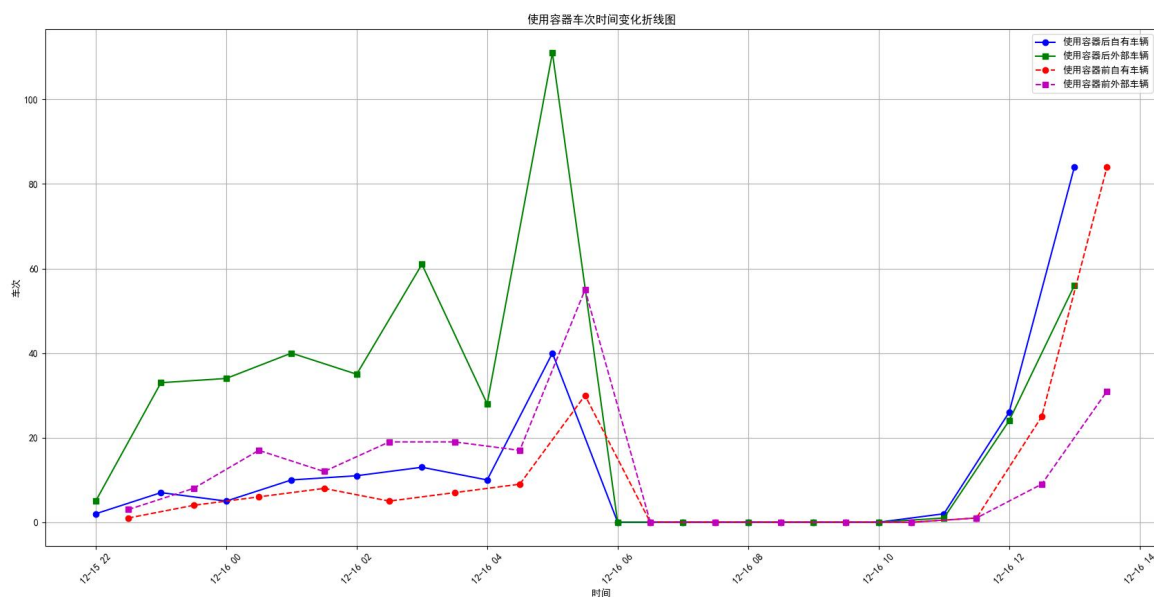


图 14

图 14 为车次随时间变化的折线图，在问题 2 图 10 的基础上加入问题三车次随时间变化的折线图，讨论引入容器后车次变化。该图显示，使用容器后自有车辆和外部车辆的车次变化趋势与使用前存在显著差异。凌晨 22 点至 02 点，车次逐渐上升，外部车辆增幅更大，02 点至 04 点达小高峰。06 点至 08 点，车次骤降，接近零。12 点至 14 点，车次再次上升，自有车辆增幅明显。对比发现，使用容器后，车次变化幅度整体增大，且在 12 点至 14 点增幅更显著，表明容器使用优化了运输流程，提升了运输效率。总体来看，自有车辆车次变化幅度大于外部车辆，且使用容器后这一趋势进一步加强。

（四）问题四

1. 模型建立

（1）蒙特卡洛模拟

蒙特卡洛(MonteCarlo)模拟是一个极具通用性的工具，它将确定性的原始数据看作一组随机变量的真值，通过引入“噪声”或“扰动”来构造一个随机样本空间，从而在虚拟实验中反复采样，统计并近似真实系统的输出分布和统计特性。预测偏差模型基于蒙特卡洛模拟方法，通过生成多种可能的偏差场景，评估货量预测偏差对车辆调度优化结果的影响。该模型将偏差类型进行系统分类，并针对每种类型设定不同参数，对整个调度流程进行重新计算，从而获得关键性能指标的变化。

(2) 鲁棒性评估指标体系

我们设定以下三类核心指标以此来全面评价调度模型的鲁棒性：

表 4

指标名称	定义	公式
β_1	平均自有车周转率	$\beta_1 = \frac{\sum_{i=1}^G Z_1}{G}$
β_2	平均车辆均包裹	$\beta_2 = \frac{\sum_{i=1}^G Z_2}{G}$
β_3	平均总成本	$\beta_3 = \frac{\sum_{i=1}^G Z_3}{G}$

注：鲁棒性越高各项指标变化越小

(3) 输入与参数定义

①输入数据

- 1) 问题一预测的每条线路的货量： Q_{ic}
- 2) 车队信息：自有车辆数 V 、各线路在途时间： T_j 、变动成本 C_j
- 3) 标准容器使用决策（问题三结果）：每条需求是否使用容器（布尔标值）

②参数

- 1) 扰动幅度： d ，当 $d = 0.1$ 表示 $\pm 10\%$ 波动)
- 2) 蒙特卡洛样本数： N
- 3) 车辆装载量： W （标准容器：800，普通：1000）
- 4) 时间参数
 - a. 装车时间：标准容器： $\hat{U}/2$ ，普通： $U/2$
 - b. 卸车时间：标准容器： $\hat{U}/2$ ，普通： $U/2$
 - c. 发运节点：6:00 和 14:00

(4) 蒙特卡洛模型建立

对每个样本 λ ，生成所有线路和时间段的扰动货量：

$$Q_{ic}^{\lambda} = \max(Q_{ic} \cdot (1 + \delta_{ic}^{\lambda}), 0), \quad \delta_{ic}^{\lambda} \sim U(-d, d) \quad (4.1)$$

其中： $\lambda = 1, 2, \dots, N$

根据问题 3 的结果，计算出每辆车的装载量 W ，对每条线路或串点组按时间顺序累计货量，计算出出发运时间：

$$N_{\rho}^{i\lambda} = \left\lceil \frac{\sum_c Q_{ic}^{\lambda}}{W} \right\rceil, \quad W \in \{800, 1000\} \quad (4.2)$$

ρ ：代表车次

W ：每辆车的装载量

λ ：蒙特卡洛模型的样本点

(5) 指标计算

①自有车周转率：

$$Z_1^{\lambda} = \frac{\sum_{jk} X_{jk}}{V_d} \quad (4.3)$$

②车辆均包裹：

$$Z_2^{\lambda} = \frac{\sum_j Q_j}{\sum_v u_v + \sum_j y_j} \quad (4.4)$$

③总成本：

$$Z_3^{\lambda} = \sum_v u_v \cdot S + \sum_v \sum_{j \in v} X_{jk} \cdot C_j + \sum_j y_j \cdot e_j \quad (4.5)$$

2. 模型结果分析

(1) 敏感性分析

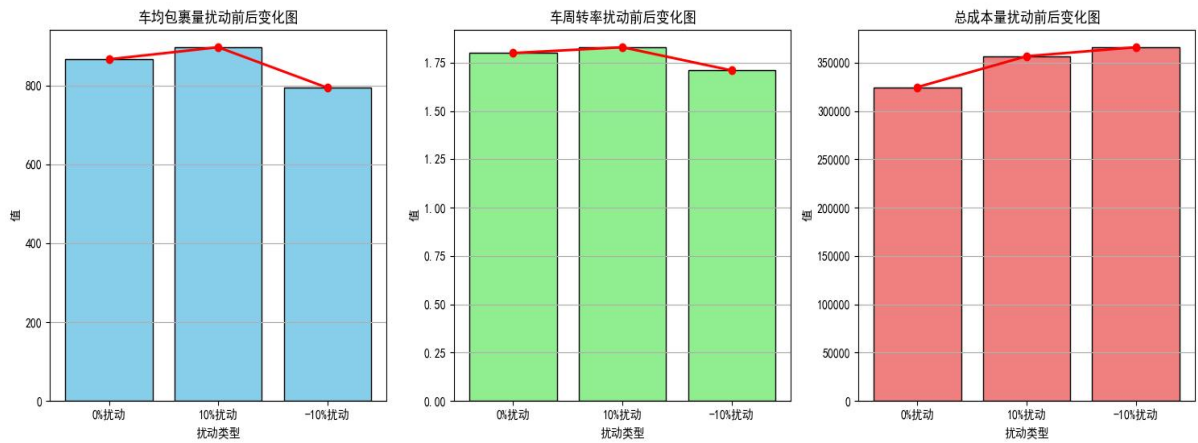


图 15

图 15 分析结果表明，车均包裹量、车周转率和总成本量对扰动的响应各异。车均包裹量随扰动增加而上升，扰动为 10% 时增幅明显，扰动为 -10% 时则下降。车周转率在 0% 和 10% 扰动下保持稳定，但在 -10% 扰动时显著降低。总成本量对扰动最为敏感，随着扰动增加，成本持续上升，且在 -10% 扰动时增幅更为显著。这反映出总成本量受扰动影响最大，系统需重点关注成本控制以应对不确定性。

(2) 鲁棒性评估

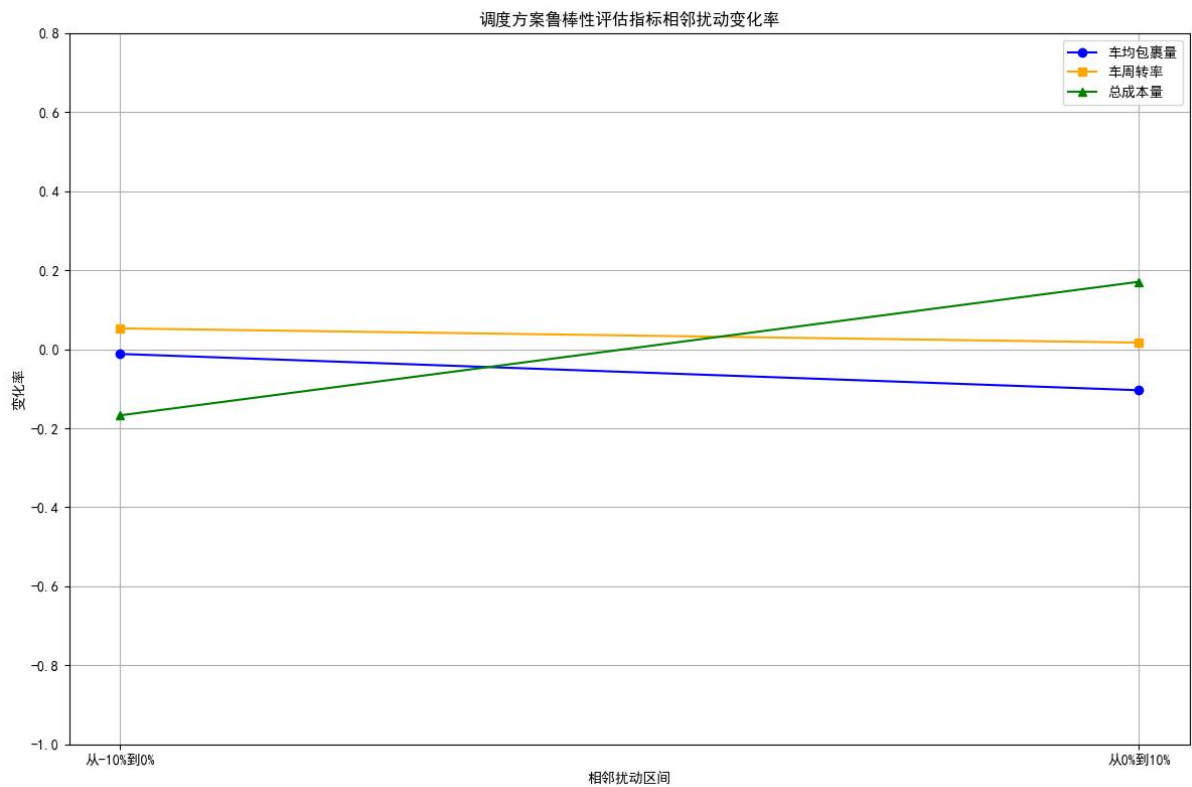


图 16

图 16 为调度方案鲁棒性评估指标相邻扰动变化率，由图可知，随着扰动从负区间向正区间变化，车均包裹量、车周转率、总成本量的变化率仅在 -0.15% 至 0.15% 间变化，因此，此模型具有较高平的抗扰动性和适应性，可以应对预测货量的突发性偏差、时间分布偏差和整体偏差对我们的调度模型优化结果的影响。

六、模型评价与推广

（一）模型的评价

对上面建立的数学模型进行综合评估，目的是展现模型在处理特定问题时的表现及可能存在的不足。方便对所建立的数学模型有更深入的了解，以及提出建议和模型改进方案。

1. 时间序列模型 ARIMA

优点：问题一中的时间序列模型 ARIMA 能够捕捉历史货量数据的趋势和季节性，适合对各条线路货量的短期预测。

缺点：线性假设限制无法处理非线性波动的货量数据，如突发事件造成的货量激增。

数据需求高：需要大量历史数据，这样预测出来的货量结果准确性才高。此外这种模型对新兴线路或数据稀疏场景效果差。。只适合静态参数预测：模型参数固定，难以动态适应数据分布变化。

2. 多目标优化模型

在问题二和问题三我们使用多目标优化模型求解各条线路的调度方案，并取得较好调度的结果。

优点：

综合权衡：可同时优化周转率、均包裹、总成本目标，提供全局最优解。

灵活性强：通过权重调整（ ω_1 ， ω_2 ， ω_3 ）适配不同目标关注度。

缺点：

计算复杂度高：变量和约束条件多时，模型求解时间按指数级增长。

权重主观性：权重设定依赖经验，可能偏离实际需求。

局部最优风险：非凸问题中易陷入局部最优解。

3. 贪心算法模型

在求解多目标优化模型中，我们使用贪心算法求解最优的调度方案。

优点：

计算高效：实时性强，适合实时调度，快速生成可行解。

简单且易于实现：无需复杂数学模型，适用于解决小规模问题。

缺点：

短视决策：局部最优导致全局次优，如车辆过早占用影响后续任务。

依赖排序策略：发运时间排序可能忽略任务间潜在协同性。

4. 基于蒙特卡洛的预测值扰动模型

在问题四中，我们基于蒙特卡洛的预测值扰动模型来评估系统的鲁棒性。

优点：

风险量化：通过随机采样评估不确定性，提供概率化结果（如成本分布）。

通用性强：适用于多种扰动类型，如货量偏差、时间延迟等。

缺点：

所需数据量大：需大量样本输入从而保证精度，计算耗时长。

结果解释复杂：概率分布需结合业务知识解读，决策门槛高。

（二）创新点

问题 1 将校正因子与时间分布结合，提高了预测的准确性，尤其是针对特定线路的预测结果。

问题 2 使用了贪心算法进行车辆调度，将多目标优化与贪心算法结合，通过动态调整车辆分配策略，有效平衡了不同目标之间的冲突，提高了车辆利用率。

问题 3 引入了标准容器，灵活应对不同运输工具的特性，通过动态调整发运次数和车辆调度策略，进一步优化运输效率。

问题 4 将蒙特卡洛模拟与调度模型结合，量化了预测不确定性对整体系统的影响，为实际运营提供了风险管理的依据。

在整个过程中，可能还有一些跨问题的创新点，例如多目标优化的动态权重调整、串点任务合并规则的灵活性、以及预测模型与实际调度的协同优化。需要逐一分析每个步骤中使用的模型，明确其创新之处。

以上创新方法通过**动态调整机制**、**混合算法策略**、**弹性规则设计**及**闭环反馈优化**，显著提升了短途运输调度模型的多目标协同能力、抗干扰性和实际适用性。

（三）模型的改进

为进一步提高模型的表现，可以考虑引入更多的动态因素来优化决策过程。例如，在预测货量时，增加对突发事件的敏感性，如极端天气、突发性促销。结合实时数据进行调整、以提高预测的准确性。同时，调度部分可以更加灵活地应对实际运营中的变化，比如引入更智能的容器使用决策机制，动态选择是否使用标准容器，依据实时的货量和运输情况进行调整。此外，针对模型复杂的

计算，可以尝试优化算法，减少计算资源的消耗，尤其是在大规模调度任务中，以提升效率和适应性。通过这些改进，模型不仅能更好地应对实际情况中的变化，还能在实际操作中更加高效、灵活地执行任务。

1. 时间序列模型 ARIMA

优化方案：引入 LSTM 神经网络

理由：LSTM 可捕捉非线性时序特征（如促销导致的货量激增），通过记忆单元处理突发波动。

对比优势：相比 ARIMA 的线性假设，LSTM 在复杂场景降低预测误差。

2. 多目标优化模型

优化方案：动态权重调整

理由：在货量高峰期周转率最大化优先（ $\omega_1 \uparrow$ ），低谷期总成本最小化优先（ $\omega_3 \uparrow$ ），适配不同时段需求。

对比优势：相比固定权重，可调高自有车辆利用率，降低外部车辆承运成本。

3. 贪心算法模型

优化方案：混合优化策略（贪心+遗传算法）

理由：由贪心算法生成初始解，遗传算法通过交叉、变异优化任务顺序，提升全局效率。

对比优势：可进一步提高自有车辆周转率。

4. 基于蒙特卡洛的预测值扰动模型

优化方案：重要性采样（Importance Sampling）：

理由：针对高频扰动线路增加采样权重，提升评估效率。

对比优势：在精度变化不大的情况下，减少采样所需样本数，减少计算时间。

优化方案：实时增量模拟

理由：仅对当日预测偏差显著的线路进行局部重模拟，而非全量计算。

对比优势：减少响应时间，支持动态调度。

（四）模型推广

本文所建立的模型通过"动态预测-智能调度-鲁棒评估"三位一体改进，具备行业推广价值。短期可快速降本增效，长期支撑智慧物流升级。该模型可推广到电商物流、冷链配送、城市货运等短途

运输场景，尤其适合多网点、高频次运输需求。还可扩展至跨境物流的"最后一公里"分拣中心到边境仓的调度。在信息化方面，可与高德地图、菜鸟网络合作，获取实时路况与网点数据，优化在途时长预测。

七、参考文献

- [1]吕浩,吕志.ARIMA 时序模型在广东省经济动态分析中的创新探索[J].特区经济,2025,(03):119-122.
- [2]刘静,赵瑞青,赵执扬,等.基于经验模态分解的 ARIMA 模型在山西省肺结核预测中的应用[J/OL].中国卫生统计,1-5[2025-04-20].<http://kns.cnki.net/kcms/detail/21.1153.R.20250317.1041.004.html>.
- [3]张茜,聂亿兵,董存军.基于贪心算法与动态规划结合的线路优化研究[J].现代商贸工业,2025,(07):236-238.DOI:10.19311/j.cnki.1672-3198.2025.07.076.
- [4]王传旺,韩伟力.面向子词级口令猜测模型的蒙特卡洛强度评估优化算法[J].计算机应用与软件,2025,42(02):361-366+373.
- [5]周贤来.基于贪心算法的 5G 网络通信资源调度方法[J].长江信息通信,2024,37(11):199-202.DOI:10.20153/j.issn.2096-9759.2024.11.060.
- [6]仲毅.山东省区域物流发展水平评价及物流网络结构研究[D].青岛科技大学,2024.DOI:10.27264/d.cnki.gqdhc.2024.001240.
- [7]李俊成,王巍.基于混合优化算法的多目标装配线平衡问题求解模型[J].中国新技术新产品,2024,(01):142-145.DOI:10.13612/j.cnki.cntp.2024.01.044.
- [8]国务院办公厅关于印发“十四五”现代物流发展规划的通知[J].中华人民共和国国务院公报,2023,(01):47-60.

八、附录

本次我们使用 **Pycharm** 编程软件完成所有数据处理、图表生成、表格填入工作，具体源码见下：

①三维货量分布图 python 源码

```
# 三维货量分布图
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
from datetime import datetime, timedelta

# 设置中文字体支持（确保系统安装了中文字体）
plt.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体显示中文
plt.rcParams['axes.unicode_minus'] = False # 正确显示负号

# 读取Excel文件中的数据（请确保路径正确）
df = pd.read_excel(r'H:\mathcup\附件2.xlsx') # 使用原始字符串避免转义问题

# 转换日期和时间格式
df['日期'] = pd.to_datetime(df['日期'], format='%Y-%m-%d')
df['分钟起始'] = df['分钟起始'].astype(str).str.zfill(6) # 确保时间格式统一

# 拼接完整的日期时间
df['日期时间'] = df['日期'].dt.strftime('%Y-%m-%d') + ' ' + df['分钟起始']
df['日期时间'] = pd.to_datetime(df['日期时间'], format='%Y-%m-%d %H:%M:%S')

# 筛选指定时间范围的数据
start_time = datetime(year=2024, month=11, day=30, hour=21, minute=0)
end_time = datetime(year=2024, month=12, day=15, hour=21, minute=0)
mask = (df['日期时间'] >= start_time) & (df['日期时间'] <= end_time)
filtered_df = df.loc[mask]

# 提取线路编码并去重排序
route_codes = filtered_df['线路编码'].sort_values().unique()

# 获取唯一的时间点并排序
time_points = np.sort(filtered_df['日期时间'].unique())
```

```

# 获取唯一的时间点并排序
time_points = np.sort(filtered_df['日期时间'].unique())

# 创建网格数据
time_indices = np.arange(len(time_points))
route_indices = np.arange(len(route_codes))

time_grid, route_grid = np.meshgrid(*xi: time_indices, route_indices)

# 初始化包裹量网格
package_grid = np.zeros((len(route_codes), len(time_points)))

# 填充包裹量数据
for i, route in enumerate(route_codes):
    for j, time_point in enumerate(time_points):
        package_count = filtered_df[
            (filtered_df['线路编码'] == route) &
            (filtered_df['日期时间'] == time_point)
        ]['包裹量'].sum()
        package_grid[i, j] = package_count

# 创建3D图形
fig = plt.figure(figsize=(16, 10))
ax = fig.add_subplot(111, projection='3d')

# 绘制曲面图（使用更合适的颜色映射）
surf = ax.plot_surface(
    time_grid,
    route_grid,
    package_grid,
    cmap='viridis', # 更清晰的颜色映射
    edgecolor='none',
    alpha=0.8 # 略微调整透明度
)

# 添加颜色条
fig.colorbar(
    surf,
    shrink=0.5,
    aspect=10,
    label='累计包裹量',
    pad=0.1 # 调整颜色条位置
)

# 设置标签（中文显示）
ax.set_xlabel('时间', fontsize=12, fontweight='bold')
ax.set_ylabel('线路编码', fontsize=12, fontweight='bold')
ax.set_zlabel('累计包裹量', fontsize=12, fontweight='bold')

plt.show()

```

②附件二数据预处理 python 源码

```
import pandas as pd

# 读取 Excel 文件
df = pd.read_excel(io: 'H:\\mathcup\\附件2.xlsx', engine='openpyxl')

routes = df.iloc[:, 0]
timestamps = df.iloc[:, 1]
parcel_counts = df.iloc[:, 3]

# 提取日期信息
df['日期'] = pd.to_datetime(timestamps).dt.date

df = df[df['日期'] != pd.Timestamp('2024-11-30').date()]

# 添加线路信息到 DataFrame
df['线路'] = routes

# 按线路和日期分组并求和
daily_parcel_counts = df.groupby(['线路', '日期'])['包裹量'].sum().reset_index()

# 保存结果
daily_parcel_counts.to_excel('H:\\mathcup\\每日每线路包裹量.xlsx', index=False)
```

③均方根计算 python 源码

```
import pandas as pd
import numpy as np

# 读取两个 Excel 文件
file1 = 'H:\\mathcup\\每日每线路包裹量.xlsx' # 替换第一个 Excel 文件路径
file2 = 'H:\\mathcup\\包裹量比值结果&k.xlsx' # 替换为第二个 Excel 文件路径

df1 = pd.read_excel(file1)
df2 = pd.read_excel(file2)

column1 = df1['包裹量']
column2 = df2['预测包裹量']

mean_actual = np.mean(column1) # 计算实际数据均值
# 确保两个列的长度相同
if len(column1) != len(column2):
    print("两个列的长度不一致，无法进行比较。")
else:
    # 计算均方根误差
    mse = np.mean((column1 - column2) ** 2)
    rmse = np.sqrt(mse)
    print("均方根误差 (RMSE):", rmse)
    rrmse=rmse/mean_actual*100
    print("rrms:", rrmse, "%")
```

```

import pandas as pd
# 读取 Excel 文件
new_file_path = "D:new.xlsx"
attachment3_file_path = "D:附件3.xlsx"

new_df = pd.read_excel(new_file_path)
attachment3_df = pd.read_excel(attachment3_file_path)

merged_df = pd.merge(new_df, attachment3_df, on=["线路编码", "日期"], suffixes=('_new', '_attachment3'))

# 计算差异 = 实际货量 - 预知货量
merged_df['差异'] = merged_df['包裹量_new'] - merged_df['包裹量_attachment3']

# 计算未下单货量和取消货量
merged_df['未下单货量'] = merged_df['差异'].apply(lambda x: x if x > 0 else 0)
merged_df['取消货量'] = merged_df['差异'].apply(lambda x: abs(x) if x < 0 else 0)

# 计算每个线路编码的每天包裹量的平均值 (过去15天)
route_date_avg = merged_df.groupby("线路编码")["包裹量_attachment3"].mean().reset_index()
route_date_avg.columns = ["线路编码", "日均包裹量"]

merged_df = pd.merge(merged_df, route_date_avg, on="线路编码")

merged_df['未下单率'] = merged_df['未下单货量'] / merged_df['日均包裹量']
merged_df['取消率'] = merged_df['取消货量'] / merged_df['日均包裹量']

result_df = merged_df[["线路编码", "日期", "包裹量_new", "包裹量_attachment3", "差异", "未下单货量", "取消货量", "未下单率",
result_df.to_excel("D:订单分析结果.xlsx", index=False)]

```

```

1 import pandas as pd
2
3 new_file_path = "D:附件3.xlsx"
4 new_df = pd.read_excel(new_file_path)
5
6 # 计算总包裹量的均值和标准差
7 mean_packages = new_df["包裹量"].mean()
8 std_packages = new_df["包裹量"].std()
9
10 print(f"总包裹量的均值: {mean_packages}")
11 print(f"总包裹量的标准差: {std_packages}")
12
13 file_path = "D:附件3.xlsx"
14 df = pd.read_excel(file_path, dtype={"包裹量": int})
15
16 # 筛选条件: 附件3的包裹量与均值的绝对值大于3倍标准差
17 filtered_df = df[
18     abs(df["包裹量"] - mean_packages) <= 3 * std_packages
19 ]
20
21 # 保存筛选后的数据到新的 Excel 文件
22 filtered_df.to_excel("D:new_data_1.xlsx", index=False)
23

```



```

1 import pandas as pd
2
3 new_file_path = "D:附件3.xlsx"
4 new_df = pd.read_excel(new_file_path)
5
6 # 计算总包裹量的均值和标准差
7 mean_packages = new_df["包裹量"].mean()
8 std_packages = new_df["包裹量"].std()
9
10 print(f"总包裹量的均值: {mean_packages}")
11 print(f"总包裹量的标准差: {std_packages}")
12
13 file_path = "D:附件3.xlsx"
14 df = pd.read_excel(file_path, dtype={"包裹量": int})
15
16 # 筛选条件: 附件3的包裹量与均值的绝对值大于3倍标准差
17 filtered_df = df[
18     abs(df["包裹量"] - mean_packages) <= 3 * std_packages
19 ]
20
21 # 保存筛选后的数据到新的 Excel 文件
22 filtered_df.to_excel("D:new_data_1.xlsx", index=False)
23

```

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 import numpy as np
5 from scipy.stats import norm
6
7 file_path = "D:new_data_1.xlsx"
8 df = pd.read_excel(file_path)
9
10 packages = df["包裹量"]
11
12 mean_packages = packages.mean()
13 std_packages = packages.std()
14
15 # 绘制直方图和正态分布曲线
16 plt.figure(figsize=(10, 6))
17 sns.histplot(packages, bins=30, kde=False, color='skyblue', alpha=0.6, stat='density')
18
19 # 生成正态分布曲线
20 x = np.linspace(packages.min(), packages.max(), num=100)
21 p = norm.pdf(x, mean_packages, std_packages)
22 plt.plot(*args: x, p, 'k', linewidth=2)
23
24 # 添加标题和标签
25 plt.title('正态分布图\n均值: {:.2f}, 标准差: {:.2f}'.format(*args: mean_packages, std_packages))
26 plt.xlabel('包裹量')
27 plt.ylabel('概率密度')
28
29 plt.show()

```

```

1  import pandas as pd
2  import matplotlib.pyplot as plt
3  import numpy as np
4  import matplotlib as mpl
5  from scipy.stats import norm
6  from scipy.interpolate import UnivariateSpline
7
8  # 设置中文字体
9  mpl.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体
10 mpl.rcParams['axes.unicode_minus'] = False # 正确显示负号
11
12 # 读取 Excel 文件
13 file_path = 'D:包裹量比值结果&k.xlsx' # 替换为你的文件路径
14 df = pd.read_excel(file_path)
15
16 # 提取包裹量_attachment3 列的数据
17 data = df['包裹量_attachment3']
18
19 # 将数据归一化到 [0, 1] 范围内
20 normalized_data = (data - np.min(data)) / (np.max(data) - np.min(data))
21
22 # 设置直方图的区间
23 plt.figure(figsize=(12, 6))
24 plt.hist(normalized_data, bins=20, color='skyblue', edgecolor='black', alpha=0.7, density=True, label='归一化直方图')
25
26 # 计算每个区间的中点
27 counts, bin_edges = np.histogram(normalized_data, bins=20, density=True)
28 bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
29
30 # 使用UnivariateSpline生成归一化平滑曲线
31 spline = UnivariateSpline(bin_centers, counts, s=0.5) # s控制平滑程度
32 x_smooth = np.linspace(bin_edges[0], bin_edges[-1], num=1000)
33 y_smooth = spline(x_smooth)
34
35 # 绘制平滑曲线
36 plt.plot(*args: x_smooth, y_smooth, color='mediumblue', linewidth=2, linestyle='--', label='平滑曲线')
37 |
38 plt.title('包裹量的归一化分布直方图')
39 plt.xlabel('归一化包裹量 (0到1)')
40 plt.ylabel('概率密度')
41
42 # 设置 x 轴刻度范围，左右增加空白区域
43 plt.xlim(*args: -0.1, 1.1) # 左右各留出10%的空白区域
44
45 # 添加图例和网格
46 plt.legend(fontsize=12)
47 plt.grid(visible=True, linestyle='--', alpha=0.7)
48
49 plt.tight_layout()
50 plt.show()

```



```

1 import pandas as pd
2 from statsmodels.tsa.arima.model import ARIMA
3 import numpy as np
4
5 # 指定线路编码列表
6 route_codes = [
7     "场地1 - 站点1 - 1400", "场地1 - 站点1 - 0600", "场地1 - 站点2 - 1400",
8     "场地1 - 站点3 - 1400", "场地1 - 站点3 - 0600", "场地1 - 站点4 - 1400",
9     "场地1 - 站点4 - 0600", "场地1 - 站点5 - 1400", "场地1 - 站点5 - 0600",
10    "场地1 - 站点6 - 1400", "场地1 - 站点6 - 0600", "场地1 - 站点7 - 1400",
11    "场地1 - 站点7 - 0600", "场地1 - 站点8 - 1400", "场地1 - 站点9 - 1400",
12    "场地1 - 站点9 - 0600", "场地1 - 站点10 - 0600", "场地1 - 站点10 - 1400",
13    "场地1 - 站点11 - 1400", "场地1 - 站点11 - 0600", "场地1 - 站点12 - 1400",
14    "场地1 - 站点13 - 1400", "场地1 - 站点13 - 0600", "场地1 - 站点12 - 0600",
15    "场地1 - 站点8 - 0600", "场地1 - 站点14 - 1400", "场地1 - 站点14 - 0600",
16    "场地1 - 站点15 - 1400", "场地1 - 站点15 - 0600", "场地1 - 站点16 - 1400",
17    "场地1 - 站点16 - 0600", "场地1 - 站点17 - 1400", "场地1 - 站点17 - 0600",
18    "场地1 - 站点18 - 1400", "场地1 - 站点18 - 0600", "场地1 - 站点19 - 1400",
19    "场地1 - 站点20 - 1400", "场地1 - 站点20 - 0600", "场地1 - 站点19 - 0600",
20    "场地1 - 站点21 - 1400", "场地1 - 站点22 - 1400", "场地1 - 站点22 - 0600",
21    "场地1 - 站点2 - 0600", "场地1 - 站点23 - 1400", "场地1 - 站点23 - 0600",
22    "场地1 - 站点24 - 1400", "场地1 - 站点24 - 0600", "场地1 - 站点21 - 0600",
23    "场地1 - 站点25 - 1400", "场地1 - 站点25 - 0600", "场地1 - 站点26 - 1400",
24    "场地1 - 站点26 - 0600", "场地1 - 站点27 - 1400", "场地1 - 站点27 - 0600",
25    "场地1 - 站点28 - 1400", "场地1 - 站点28 - 0600", "场地1 - 站点29 - 0600",
26    "场地1 - 站点30 - 1400", "场地1 - 站点30 - 0600", "场地1 - 站点31 - 1400",
27    "场地1 - 站点31 - 0600", "场地1 - 站点32 - 1400", "场地1 - 站点32 - 0600",
28    "场地1 - 站点33 - 1400", "场地1 - 站点33 - 0600", "场地1 - 站点34 - 1400",
29    "场地1 - 站点34 - 0600", "场地1 - 站点29 - 1400", "场地1 - 站点35 - 1400",
30    "场地1 - 站点35 - 0600", "场地1 - 站点36 - 1400", "场地1 - 站点36 - 0600",
31    "场地2 - 站点37 - 1400", "场地2 - 站点37 - 0600", "场地2 - 站点38 - 1400",
32    "场地2 - 站点38 - 0600", "场地2 - 站点39 - 1400", "场地2 - 站点39 - 0600",
33    "场地2 - 站点40 - 1400", "场地2 - 站点40 - 0600", "场地2 - 站点41 - 1400",
34    "场地2 - 站点41 - 0600", "场地2 - 站点42 - 1400", "场地2 - 站点42 - 0600",
35    "场地2 - 站点43 - 1400", "场地2 - 站点43 - 0600", "场地2 - 站点44 - 1400",
36    "场地2 - 站点44 - 0600", "场地2 - 站点45 - 1400", "场地2 - 站点45 - 0600",
37    "场地2 - 站点46 - 1400", "场地2 - 站点47 - 1400", "场地2 - 站点48 - 1400",
38    "场地2 - 站点49 - 1400", "场地2 - 站点49 - 0600", "场地2 - 站点46 - 0600",
39    "场地2 - 站点48 - 0600", "场地2 - 站点47 - 0600", "场地2 - 站点50 - 1400",
40    "场地2 - 站点50 - 0600", "场地2 - 站点51 - 1400", "场地2 - 站点51 - 0600",
41    "场地2 - 站点53 - 1400", "场地2 - 站点53 - 0600", "场地2 - 站点52 - 0600",
42    "场地2 - 站点51 - 0600", "场地2 - 站点54 - 1400", "场地2 - 站点54 - 0600",
43    "场地2 - 站点55 - 1400", "场地2 - 站点55 - 0600", "场地3 - 站点56 - 1400",
44    "场地3 - 站点57 - 0600", "场地3 - 站点57 - 1400", "场地3 - 站点58 - 0600",
45    "场地3 - 站点58 - 0600", "场地3 - 站点59 - 1400", "场地3 - 站点59 - 0600",
46    "场地3 - 站点60 - 1400", "场地3 - 站点61 - 0600", "场地3 - 站点62 - 0600",
47    "场地3 - 站点63 - 1400", "场地3 - 站点61 - 1400", "场地3 - 站点60 - 0600",
48    "场地3 - 站点64 - 1400", "场地3 - 站点63 - 0600", "场地3 - 站点65 - 1400",
49    "场地3 - 站点65 - 1400", "场地3 - 站点67 - 1400", "场地3 - 站点67 - 0600",
50    "场地3 - 站点64 - 0600", "场地3 - 站点65 - 0600", "场地3 - 站点68 - 1400",
51    "场地3 - 站点66 - 0600", "场地3 - 站点69 - 1400", "场地3 - 站点69 - 0600",
52    "场地3 - 站点70 - 1400", "场地3 - 站点71 - 1400", "场地3 - 站点72 - 1400",
53    "场地3 - 站点73 - 1400", "场地3 - 站点73 - 0600", "场地3 - 站点74 - 1400",
54    "场地3 - 站点74 - 0600", "场地3 - 站点75 - 1400", "场地3 - 站点75 - 0600",
55    "场地3 - 站点76 - 1400", "场地3 - 站点62 - 1400", "场地3 - 站点77 - 1400",
56    "场地3 - 站点78 - 1400", "场地3 - 站点78 - 0600", "场地3 - 站点79 - 1400",
57    "场地3 - 站点73 - 0600", "场地3 - 站点80 - 1400", "场地3 - 站点58 - 1400",
58    "场地3 - 站点81 - 0600", "场地3 - 站点76 - 0600", "场地3 - 站点81 - 0600",
59    "场地3 - 站点82 - 0600", "场地3 - 站点83 - 0600", "场地3 - 站点71 - 0600",
60    "场地3 - 站点82 - 1400", "场地3 - 站点79 - 0600", "场地3 - 站点84 - 1400",
61    "场地3 - 站点80 - 0600", "场地3 - 站点85 - 1400", "场地3 - 站点85 - 0600",
62    "场地3 - 站点86 - 0600", "场地3 - 站点87 - 1400", "场地3 - 站点87 - 1400",
63    "场地3 - 站点87 - 0600", "场地3 - 站点86 - 0600", "场地3 - 站点88 - 1400",
64    "场地3 - 站点89 - 1400", "场地3 - 站点89 - 0600", "场地3 - 站点84 - 0600",
65    "场地3 - 站点90 - 1400", "场地3 - 站点90 - 0600", "场地3 - 站点91 - 0600",
66    "场地3 - 站点88 - 0600", "场地3 - 站点91 - 1400", "场地3 - 站点92 - 0600",
67    "场地3 - 站点92 - 0600", "场地3 - 站点93 - 0600", "场地3 - 站点92 - 1400",
68    "场地3 - 站点83 - 1400", "场地3 - 站点94 - 1400", "场地3 - 站点94 - 0600",
69    "场地3 - 站点95 - 1400", "场地3 - 站点95 - 0600", "场地3 - 站点93 - 1400",
70    "场地3 - 站点96 - 1400", "场地3 - 站点96 - 0600", "场地3 - 站点97 - 0600",
71    "场地3 - 站点97 - 1400", "场地3 - 站点98 - 1400", "场地3 - 站点98 - 0600",
72    "场地3 - 站点99 - 1400", "场地3 - 站点99 - 0600", "场地4 - 站点100 - 1400",
73    "场地4 - 站点101 - 1400", "场地4 - 站点101 - 0600", "场地4 - 站点100 - 0600",
74    "场地4 - 站点102 - 0600", "场地4 - 站点103 - 0600", "场地4 - 站点103 - 1400",
75    "场地4 - 站点104 - 1400", "场地4 - 站点104 - 0600", "场地4 - 站点105 - 1400",
76    "场地4 - 站点105 - 0600", "场地4 - 站点106 - 1400", "场地4 - 站点106 - 0600",
77    "场地4 - 站点107 - 1400", "场地4 - 站点107 - 0600", "场地4 - 站点108 - 0600",
78    "场地4 - 站点109 - 1400", "场地4 - 站点109 - 0600", "场地4 - 站点110 - 1400",
79    "场地4 - 站点111 - 1400", "场地4 - 站点111 - 0600", "场地4 - 站点112 - 1400",
80    "场地4 - 站点110 - 0600", "场地4 - 站点112 - 0600", "场地4 - 站点108 - 1400",
81    "场地4 - 站点113 - 1400", "场地4 - 站点113 - 0600", "场地4 - 站点102 - 1400",
82    "场地4 - 站点114 - 1400", "场地4 - 站点114 - 0600", "场地4 - 站点115 - 0600",
83    "场地4 - 站点116 - 1400", "场地4 - 站点116 - 0600", "场地4 - 站点117 - 1400",
84    "场地4 - 站点117 - 0600", "场地4 - 站点118 - 1400", "场地4 - 站点118 - 0600",
85    "场地4 - 站点119 - 1400", "场地4 - 站点119 - 0600", "场地4 - 站点120 - 1400",
86    "场地4 - 站点120 - 0600", "场地4 - 站点115 - 1400", "场地4 - 站点121 - 1400",
87    "场地4 - 站点121 - 0600", "场地4 - 站点122 - 1400", "场地4 - 站点122 - 0600",
88    "场地4 - 站点123 - 0600", "场地4 - 站点124 - 0600", "场地4 - 站点125 - 1400",
89    "场地4 - 站点125 - 0600", "场地4 - 站点126 - 1400", "场地4 - 站点126 - 0600",
90    "场地4 - 站点127 - 1400", "场地4 - 站点127 - 0600", "场地4 - 站点124 - 1400",
91    "场地4 - 站点128 - 1400", "场地4 - 站点128 - 0600", "场地4 - 站点129 - 1400",
92    "场地4 - 站点129 - 0600", "场地4 - 站点123 - 1400", "场地4 - 站点130 - 1400",
93    "场地4 - 站点130 - 0600", "场地4 - 站点131 - 1400", "场地4 - 站点131 - 0600",
94    "场地4 - 站点132 - 1400", "场地4 - 站点132 - 0600", "场地4 - 站点133 - 1400",
95    "场地4 - 站点133 - 0600", "场地4 - 站点134 - 1400", "场地4 - 站点134 - 0600",
96    "场地4 - 站点135 - 1400", "场地4 - 站点135 - 0600", "场地4 - 站点136 - 1400",
97    "场地4 - 站点136 - 0600", "场地4 - 站点137 - 1400", "场地4 - 站点137 - 0600",
98    "场地4 - 站点138 - 0600", "场地4 - 站点139 - 1400", "场地4 - 站点139 - 0600",
99    "场地4 - 站点140 - 0600", "场地4 - 站点138 - 1400", "场地4 - 站点140 - 1400",
100 ]
101
102 new_df = pd.read_excel("D:new.xlsx")
103 attachment3_df = pd.read_excel("D:附件3.xlsx")
104
105 merged_df = pd.merge(new_df, attachment3_df, on=["线路编码", "日期"], suffixes=('_new', '_attachment3'))
106
107 # 创建预测结果列表
108 forecast_results = []
109
110 # 遍历每个线路编码
111 for route_code in route_codes:
112     # 筛选特定线路编码的数据
113     route_data = merged_df[merged_df["线路编码"] == route_code]
114     route_data = route_data.sort_values(by="日期")
115
116     # 构建时间序列数据
117     route_data['日期'] = pd.to_datetime(route_data['日期'])
118     time_series_data = route_data.set_index('日期')[['包裹量_new']]
119     time_series_data = time_series_data.asfreq('D')
120     time_series_data = time_series_data.fillna(method='ffill')
121
122     # 构建 ARIMA 模型并预测
123     try:
124         model = ARIMA(time_series_data, order=(1, 1, 1))
125         model_fit = model.fit()
126         forecast = model_fit.forecast(steps=1)
127         forecast_value = forecast[0]
128     except:
129         forecast_value = np.nan # 如果模型拟合失败, 使用 NaN 填充
130
131     forecast_results.append({
132         '线路编码': route_code,
133         '日期': '2024/12/24',
134         '预测包裹量': forecast_value
135     })
136
137 forecast_df = pd.DataFrame(forecast_results)
138
139 forecast_df.to_excel(excel_writer="D:预测结果arim.xlsx", index=False)

```

```

1 import numpy as np
2 import pandas as pd
3 from datetime import datetime, timedelta
4 import heapq
5
6 # 读取附件数据
7 attachment1 = pd.read_excel(r'D:\附件1.xlsx')
8 attachment2 = pd.read_excel(r'D:\附件2.xlsx')
9 attachment3 = pd.read_excel(r'D:\附件3.xlsx')
10 package_data = pd.read_excel(r'D:\结果表2.xlsx')
11
12 # 数据预处理
13 # 合并附件1和附件3的数据，读取线路的起止点站名
14 line_site_info = pd.merge(attachment1, attachment3, left_on='目的站地', right_on='站编号1', how='left')
15 line_site_info.rename(columns={'站编号2': '可串点站名'})
16 line_site_info.drop(columns=['站编号1'], inplace=True)
17 # 读取附件2的线路编号
18 line_codes = line_site_info['线路编号'].unique()
19
20 # 定义车辆容量和固定成本
21 vehicle_capacity = 1000 # 车辆容量
22 vehicle_fixed_cost = 100 # 车辆固定成本
23
24 # 定义装车卸车时间（分钟）
25 loading_time = 45 # 装车时间
26 unloading_time = 45 # 卸车时间
27
28 # 读取当前队伍自有车辆数据
29 fleet_vehicles = dict(zip(attachment5['车队编号'], attachment5['自有车数量']))
30
31 # 初始化为每个车队的可用车辆（先队列，按可用时间排序），并为每辆车添加编号
32 available_vehicles = {}
33 for fleet in fleet_vehicles:
34     available_vehicles[fleet] = []
35     for vehicle_num in range(1, fleet_vehicles[fleet] + 1):
36         heapq.heappush(available_vehicles[fleet], (datetime(year=2024, month=12, day=15, hour=21, minute=0),
37                                                     datetime(year=2024, month=12, day=16, hour=0, minute=0),
38                                                     minutes=loading_time))
39
40 # 定义时间增量（10分钟）
41 time_increment_10min = timedelta(minutes=10)
42
43 # 用于存储调度结果的列表
44 scheduling_results = []
45
46 # 定义一个函数来获取线路的调度包裹量
47 def get_packages(line_code): # 1. 用法
48     line_package_data = package_data[package_data['线路编号'] == line_code]
49     line_package_data['时间'] = pd.to_datetime(
50         line_package_data['时间'] + line_package_data['分钟超前'].astype(str))
51     line_package_data = line_package_data.sort_values(by='时间')
52     time_intervals = line_package_data['时间'].tolist()
53     packages = line_package_data['包裹量'].tolist()
54     return time_intervals, packages
55
56 # 定义一个函数来计算每辆车的可用时间
57 def update_vehicle_available_time(vehicle_available_time, transit_time): # 2. 用法
58     return vehicle_available_time + timedelta(hours=transit_time * 2) + timedelta(minutes=unloading_time) + time_increment_10min
59
60 # 定义一个函数来检查当前时间是否在指定时间范围内
61 def is_time_in_range(start_time, end_time, check_time): # 3. 用法
62     return start_time <= check_time < end_time
63
64 # 对每一条线路进行处理
65 for line_code in line_codes:
66     line_info = line_site_info[line_site_info['线路编号'] == line_code].iloc[0]
67     fleet = line_info['车队编号']
68     transit_time = line_info['在途时长']
69     start_site = line_info['起始站地']
70     end_site = line_info['目的站地']
71     departure_time = line_info['发车时刻']
72
73     # 获取该线路的包裹数据
74     time_intervals, packages = get_packages(line_code)
75
76     accumulated_packages = 0 # 累计包裹量
77
78     # 遍历每个时间间隔的包裹量
79     for i in range(len(time_intervals)):
80         current_time = time_intervals[i]
81         accumulated_packages += packages[i]
82
83         # 判断当前时间是否在指定时间范围内
84         if is_time_in_range(datetime(year=2024, month=12, day=15, hour=21, minute=0), datetime(year=2024, month=12, day=16, hour=0, minute=0), current_time):
85             # 如果当前包裹量达到800，安排发车
86             if accumulated_packages >= 800:
87                 assigned = False
88                 if available_vehicles[fleet]:
89                     vehicle_available_time, vehicle_num = heapq.heappop(available_vehicles[fleet])
90                     time_diff = current_time - vehicle_available_time
91                     total_seconds = time_diff.total_seconds() / 60 # 计算时间差（分钟）
92                     if time_diff >= loading_time: # 如果车辆可用时间满足装车要求
93                         new_available_time = update_vehicle_available_time(current_time, transit_time)
94                         heapq.heappush(available_vehicles[fleet], (new_available_time, vehicle_num))
95                         assigned = True
96                     else:
97                         vehicle_used = f"自有车辆-{fleet}-{vehicle_num}"
98                         scheduling_results.append({
99                             '线路编号': line_code,
100                             '日期': current_time.strftime('%Y/%m/%d'),
101                             '预计发车时间': current_time.strftime('%H:%M:%S'),
102                             '发车车辆': vehicle_used,
103                             '总包裹量': min(accumulated_packages, 800) # 确保总包裹量不超过800
104                         })
105                         accumulated_packages = max(accumulated_packages - 800, 0) # 发车后重置累计包裹量，但不小于0
106
107             if not assigned: # 如果没有可用的自有车辆，则使用外部车辆
108                 vehicle_used = "外部车辆"
109                 scheduling_results.append({
110                     '线路编号': line_code,
111                     '日期': current_time.strftime('%Y/%m/%d'),
112                     '预计发车时间': current_time.strftime('%H:%M:%S'),
113                     '发车车辆': vehicle_used,
114                     '总包裹量': min(accumulated_packages, 800) # 确保总包裹量不超过800
115                 })
116                 accumulated_packages = max(accumulated_packages - 800, 0)
117
118     # 如果时间间隔指定时间段的结束，则剩发运剩余包裹
119     if i == len(time_intervals) - 1 and accumulated_packages > 0:
120         assigned = False
121         if available_vehicles[fleet]:
122             vehicle_available_time, vehicle_num = heapq.heappop(available_vehicles[fleet])
123             time_diff = current_time - vehicle_available_time
124             total_seconds = time_diff.total_seconds() / 60
125             if time_diff >= loading_time:
126                 new_available_time = update_vehicle_available_time(current_time, transit_time)
127                 heapq.heappush(available_vehicles[fleet], (new_available_time, vehicle_num))
128                 assigned = True
129             else:
130                 vehicle_used = f"自有车辆-{fleet}-{vehicle_num}"
131                 scheduling_results.append({
132                     '线路编号': line_code,
133                     '日期': current_time.strftime('%Y/%m/%d'),
134                     '预计发车时间': current_time.strftime('%H:%M:%S'),
135                     '发车车辆': vehicle_used,
136                     '总包裹量': accumulated_packages # 发运剩余包裹量
137                 })
138                 accumulated_packages = 0
139         else:
140             vehicle_used = "外部车辆"
141             scheduling_results.append({
142                 '线路编号': line_code,
143                 '日期': current_time.strftime('%Y/%m/%d'),
144                 '预计发车时间': current_time.strftime('%H:%M:%S'),
145                 '发车车辆': vehicle_used,
146                 '总包裹量': accumulated_packages # 发运剩余包裹量
147             })
148             accumulated_packages = 0
149
150 # 将结果转换为DataFrame
151 scheduling_df = pd.DataFrame(scheduling_results)
152
153 # 对调度结果按日期和线路编号进行分组（重点方案）
154 serial_point_results = []
155 grouped = scheduling_df.groupby(['日期', '预计发车时间', '发车车辆'])
156 for key, group in grouped:
157     line_code_list = group['线路编号'].tolist()
158     total_packages = group['总包裹量'].sum() # 计算总运货量
159
160     # 如果总运货量超过800，只取前3个线路并限制总运货量为800
161     if total_packages > 800:
162         sorted_group = group.sort_values(by='总包裹量', ascending=False)
163         line_code_list = sorted_group['线路编号'].tolist()
164         total_packages = 800 # 限制总运货量为800
165
166     if len(line_code_list) <= 3: # 如果线路编号的数量小于等于3，直接添加
167         serial_point_results.append({
168             '线路': ' '.join(line_code_list),
169             '日期': key[0],
170             '预计发车时间': key[1],
171             '发车车辆': key[2],
172             '总运货量': min(total_packages, 800) # 确保总运货量不超过800
173         })
174     else: # 如果线路编号的数量大于3，分组添加
175         # 计算当前组的总运货量，确保不超过800
176         current_packages = 0
177         current_lines = []
178         for idx, row in group.iterrows():
179             if current_packages + row['总包裹量'] > 800:
180                 # 如果加入当前线路会超过800，则先添加当前组
181                 serial_point_results.append({
182                     '线路': ' '.join(current_lines),
183                     '日期': key[0],
184                     '预计发车时间': key[1],
185                     '发车车辆': key[2],
186                     '总运货量': current_packages
187                 })
188                 current_packages = row['总包裹量']
189                 current_lines = [row['线路编号']]
190             else:
191                 current_packages += row['总包裹量']
192                 current_lines.append(row['线路编号'])
193
194         # 添加最后一组
195         if current_lines:
196             serial_point_results.append({
197                 '线路': ' '.join(current_lines),
198                 '日期': key[0],
199                 '预计发车时间': key[1],
200                 '发车车辆': key[2],
201                 '总运货量': current_packages
202             })
203
204 # 将结果输出到Excel
205 final_df = pd.DataFrame(serial_point_results)
206 final_df.to_excel(excel_writer=r'D:\线路调度结果.xlsx', index=False)
207
208 print("调度结果已生成并保存到 Excel 文件！")

```

```

1 import numpy as np
2 import pandas as pd
3 from datetime import datetime, timedelta
4 import heapq
5
6 attachment1 = pd.read_excel(r'D:\附件1.xlsx')
7 attachment4 = pd.read_excel(r'D:\附件4.xlsx')
8 attachment5 = pd.read_excel(r'D:\附件5.xlsx')
9 package_data = pd.read_excel(r'D:\结果表2.xlsx')
10
11 # 数据预处理
12 # 合并附件1和附件4的数据，获取线路的可串点站点
13 line_site_info = pd.merge(attachment1, attachment4, left_on='目的地', right_on='站点编号1', how='left')
14 line_site_info = line_site_info.rename(columns={'站点编号2': '可串点站点'})
15 line_site_info.drop(columns=['站点编号1'], axis=1, inplace=True)
16
17 # 获取所有线路编码
18 line_codes = line_site_info['线路编码'].unique()
19
20 # 定义车辆容量和固定成本
21 vehicle_capacity = 1000 # 车辆容量
22 vehicle_fixed_cost = 100 # 车辆固定成本
23
24 # 定义装车卸车时间（分钟）
25 loading_time = 45 # 装车时间
26 unloading_time = 45 # 卸车时间
27
28 # 读取各车队自有车辆数量
29 fleet_vehicles = dict(zip(attachment5['车队编码'], attachment5['自有车数量']))
30
31 # 初始化每个车队的可用车辆优先队列（按可用时间排序），并为每辆车添加编号
32 available_vehicles = {}
33 for fleet in fleet_vehicles:
34     available_vehicles[fleet] = []
35     for vehicle_num in range(1, fleet_vehicles[fleet] + 1):
36         heapq.heappush(available_vehicles[fleet], item=(datetime(year=2024, month=12, day=15, hour=21, minute=0), vehicle_num))
37
38 # 定义时间增量（10分钟）
39 time_increment_10min = timedelta(minutes=10)
40
41 # 用于存储调度结果的列表
42 scheduling_results = []
43
44 # 定义一个函数来获取线路的预测包裹量
45 def get_packages(line_code): # 1个用法
46     line_package_data = package_data[package_data['线路编码'] == line_code]
47     line_package_data['时间'] = pd.to_datetime(
48         line_package_data['日期'].astype(str) + ' ' + line_package_data['分钟起始'].astype(str))
49     line_package_data = line_package_data.sort_values(by='时间')
50     time_intervals = line_package_data['时间'].tolist()
51     packages = line_package_data['包裹量'].tolist()
52     return time_intervals, packages
53
54 # 定义一个函数来更新车辆可用时间
55 def update_vehicle_available_time(vehicle_available_time, transit_time): # 2用法
56     return vehicle_available_time + timedelta(hours=transit_time * 2) + timedelta(minutes=unloading_time) + time_increment_10min
57
58 # 定义一个函数来检查发运时间是否在指定时间段内
59 def is_time_in_range(start_time, end_time, check_time): # 2用法
60     return start_time <= check_time <= end_time
61
62 # 对每一条线路进行处理
63 for line_code in line_codes:
64     line_info = line_site_info[line_site_info['线路编码'] == line_code].iloc[0]
65     fleet = line_info['车队编码']
66     transit_time = line_info['在途时长']
67     start_site = line_info['起始场地']
68     end_site = line_info['目的地']
69     departure_time = line_info['发运节点']
70
71     # 获取该线路的包裹数据
72     time_intervals, packages = get_packages(line_code)
73
74     accumulated_packages = 0 # 累计包裹量
75
76     # 遍历每个时间间隔的包裹量
77     for i in range(len(time_intervals)):
78         current_time = time_intervals[i]
79         accumulated_packages += packages[i]
80
81         # 判断当前时间是否在指定时间段内
82         if is_time_in_range(datetime(year=2024, month=12, day=15, hour=21, minute=0), datetime(year=2024, month=12, day=16, hour=0, minute=0), current_time):
83             # 判断当前包裹量是否接近车辆最大容量的90%时，安排发运
84             if accumulated_packages >= vehicle_capacity * 0.9:
85                 assigned = False
86                 if available_vehicles[fleet]:
87                     vehicle_num = heapq.heappop(available_vehicles[fleet])
88                     time_diff = (current_time - vehicle_available_time).total_seconds() / 60 # 计算时间差（分钟）
89                     if time_diff >= loading_time: # 如果车辆可用时间满足装车要求
90                         new_available_time = update_vehicle_available_time(current_time, transit_time)
91                         heapq.heappush(available_vehicles[fleet], item=(new_available_time, vehicle_num))
92                         assigned = True
93                     else:
94                         heapq.heappush(available_vehicles[fleet], item=(vehicle_available_time, vehicle_num))
95
96                 if not assigned: # 如果没有可用的自有车辆，则使用外部车辆
97                     vehicle_used = "外部车辆"
98                 else:
99                     vehicle_used = f"自有车辆_{fleet}_{vehicle_num}"
100
101                 scheduling_results.append({
102                     '线路编码': line_code,
103                     '日期': current_time.strftime('%Y/%m/%d'),
104                     '预计发运时间': current_time.strftime('%H:%M:%S'),
105                     '发运车辆': vehicle_used,
106                     '发运量': accumulated_packages # 记录当前发运的发运量
107                 })
108                 accumulated_packages = 0 # 发运后重置累计包裹量
109
110 # 如果时间接近指定时间段的结束，强制发运剩余包裹
111 if i == len(time_intervals) - 1 and accumulated_packages > 0:
112     assigned = False
113     if available_vehicles[fleet]:
114         vehicle_available_time, vehicle_num = heapq.heappop(available_vehicles[fleet])
115         time_diff = (current_time - vehicle_available_time).total_seconds() / 60
116         if time_diff >= loading_time:
117             new_available_time = update_vehicle_available_time(current_time, transit_time)
118             heapq.heappush(available_vehicles[fleet], item=(new_available_time, vehicle_num))
119             assigned = True
120         else:
121             heapq.heappush(available_vehicles[fleet], item=(vehicle_available_time, vehicle_num))
122
123     if not assigned:
124         vehicle_used = "外部车辆"
125     else:
126         vehicle_used = f"自有车辆_{fleet}_{vehicle_num}"
127
128     scheduling_results.append({
129         '线路编码': line_code,
130         '日期': current_time.strftime('%Y/%m/%d'),
131         '预计发运时间': current_time.strftime('%H:%M:%S'),
132         '发运车辆': vehicle_used,
133         '发运量': accumulated_packages # 记录当前发运的发运量
134     })
135     accumulated_packages = 0
136
137 # 将结果转换为DataFrame
138 scheduling_df = pd.DataFrame(scheduling_results)
139
140 # 合并相同发运时间和车辆的线路编码（串点方案）
141 serial_point_results = {}
142 grouped = scheduling_df.groupby(['日期', '预计发运时间', '发运车辆'])
143 for key, group in grouped:
144     line_codes = group['线路编码'].tolist()
145     total_packages = group['发运量'].sum() # 计算总发运量
146     if len(line_codes) <= 3: # 如果线路编码数量小于等于3，直接添加
147         serial_point_results.append({
148             '线路': ','.join(line_codes),
149             '日期': key[0],
150             '预计发运时间': key[1],
151             '发运车辆': key[2],
152             '总发运量': total_packages
153         })
154     else: # 如果线路编码数量大于3，分组添加
155         for i in range(0, len(line_codes), 3):
156             serial_point_results.append({
157                 '线路': ','.join(line_codes_list[i:i+3]),
158                 '日期': key[0],
159                 '预计发运时间': key[1],
160                 '发运车辆': key[2],
161                 '总发运量': total_packages
162             })
163
164 final_df = pd.DataFrame(serial_point_results)
165 final_df.to_excel(excel_writer=r'D:\so hard.xlsx', index=False)
166
167

```



```

1 import numpy as np
2 import pandas as pd
3 from datetime import datetime, timedelta
4
5 attachment1 = pd.read_excel(r'D:\附件1.xlsx')
6 attachment4 = pd.read_excel(r'D:\附件4.xlsx')
7 attachment5 = pd.read_excel(r'D:\附件5.xlsx')
8
9 # 获取所有线路编码
10 line_codes = attachment1['线路编码'].unique()
11
12 # 定义车辆容量和固定成本
13 vehicle_capacity = 1000
14 vehicle_fixed_cost = 100
15
16 # 定义装车 and 卸车时间 (分钟)
17 loading_time = 45
18 unloading_time = 45
19
20 # 用于存储调度结果的字典
21 scheduling_results = {
22     '线路编码': [],
23     '预计发运时间': [],
24     '发运车辆': []
25 }
26
27 # 读取各车队自有车辆数量
28 fleet_vehicles = dict(zip(attachment5['车队编码'], attachment5['自有车辆数']))
29
30 # 将自有车辆按可用时间排序
31 available_vehicles = {}
32 for fleet in fleet_vehicles:
33     available_vehicles[fleet] = [datetime(year=2024, month=12, day=15, hour=21, minute=0)] * fleet_vehicles[fleet]
34
35 # 创建一个站点关联的字典
36 site_association = {}
37 for index, row in attachment4.iterrows():
38     site_association[row['站点编号1']] = row['站点编号2']
39
40 # 对每一条线路进行处理
41 for line_code in line_codes:
42     # 获取该线路的详细信息
43     line_info = attachment1[attachment1['线路编码'] == line_code].iloc[0]
44     start_site = line_info['起始场地']
45     end_site = line_info['目的场地']
46     dispatch_node = line_info['发运节点']
47     fleet = line_info['车队编码']
48     transit_time = line_info['在途时长'] # 单程小时数
49
50     # 计算累计包裹量 (这里简化处理, 实际应用中需要根据销量预测结果进行计算)
51     time_intervals = pd.date_range(start='2024-12-15 14:00', end='2024-12-16 14:00', freq='10min')
52     packages = np.random.randint(low=1, vehicle_capacity, size=len(time_intervals))
53
54     # 初始化车辆使用情况和发运时间
55     current_vehicles = []
56     current_time = datetime(year=2024, month=12, day=15, hour=21, minute=0)
57     accumulated_packages = 0 # 累积包裹量
58
59     # 贪心算法分配车辆
60     for i in range(len(time_intervals)):
61         if packages[i] > 0:
62             accumulated_packages += packages[i]
63
64             # 当累积包裹量接近或达到车辆容量时, 安排发车
65             if accumulated_packages >= vehicle_capacity * 0.9: # 接近1000包裹量
66                 # 尝试使用自有车辆
67                 assigned = False
68                 for idx, vehicle_time in enumerate(available_vehicles[fleet]):
69                     # 检查车辆是否可用
70                     if vehicle_time + timedelta(minutes=loading_time) + timedelta(hours=transit_time * 2) + timedelta(minutes=unloading_time) <= current_time:
71                         # 更新车辆可用时间
72                         available_vehicles[fleet][idx] = current_time + timedelta(hours=transit_time * 2) + timedelta(minutes=unloading_time)
73                         assigned = True
74                         break
75                 # 如果自有车辆不可用, 使用外部车辆
76                 if not assigned:
77                     # 记录外部车辆使用
78                     current_vehicles.append("外部车辆")
79                 else:
80                     # 记录自有车辆使用
81                     current_vehicles.append("自有车辆_" + fleet)
82
83                 # 重置累积包裹量
84                 accumulated_packages = 0
85                 # 更新当前时间
86                 current_time += timedelta(minutes=10)
87             else:
88                 # 如果当前时间间隔没有包裹, 则不需要发车
89                 current_vehicles.append("无发车")
90
91     # 确保 current_vehicles 和 time_intervals 的长度一致
92     while len(current_vehicles) < len(time_intervals):
93         current_vehicles.append("无发车")
94
95     # 记录调度结果
96     for i in range(len(time_intervals)):
97         scheduling_results['线路编码'].append(line_code)
98         scheduling_results['预计发运时间'].append(current_time)
99         scheduling_results['发运车辆'].append(current_vehicles[i])
100         # 更新时间
101         current_time += timedelta(minutes=10)
102
103 scheduling_df = pd.DataFrame(scheduling_results)
104
105 # 合并相同发运时间和车辆的线路编码 (单点方案)
106 serial_point_results = {
107     '线路': [],
108     '预计发运时间': [],
109     '发运车辆': []
110 }
111
112 grouped = scheduling_df.groupby(['预计发运时间', '发运车辆'])
113 for key, group in grouped:
114     serial_point_results['线路'].append(' - '.join(group['线路编码'].tolist()))
115     serial_point_results['预计发运时间'].append(key[0])
116     serial_point_results['发运车辆'].append(key[1])
117
118 serial_point_df = pd.DataFrame(serial_point_results)

```

```

1 import pandas as pd
2 import pulp
3 from datetime import datetime, timedelta
4
5 df_attachment1 = pd.read_excel('D:\附件1.xlsx') # 线路信息
6 df_attachment4 = pd.read_excel('D:\附件4.xlsx') # 可串点的站点集合
7 df_result_table2 = pd.read_excel('D:\结果表2.xlsx') # 预测数据
8
9 # 数据预处理
10 # 将附件1的数据转换为字典列表
11 routes = []
12 for _, row in df_attachment1.iterrows():
13     routes.append({
14         'code': row['线路编码'],
15         'origin': row['起始站地'],
16         'destination': row['目的站地'],
17         'dispatch_time': row['发运节点'],
18         'team_code': row['车队编号'],
19         'duration': row['在途时长'],
20         'own_variable_cost': row['自有变动成本'],
21         'external_cost': row['外部承运商成本']
22     })
23
24 # 将附件4的数据转换为可串点的站点集合
25 transferable_stations = {}
26 for _, row in df_attachment4.iterrows():
27     origin = row['站点编号1']
28     destination = row['站点编号2']
29     if origin not in transferable_stations:
30         transferable_stations[origin] = []
31     transferable_stations[origin].append(destination)
32
33 # 读取结果表2中的预测数据
34 # 按线路编码和日期汇总流量
35 forecast_data = {}
36 for _, row in df_result_table2.iterrows():
37     route_code = row['线路编码']
38     date = row['日期']
39     minute_start = row['分钟起始']
40     volume = row['包裹量']
41     if route_code not in forecast_data:
42         forecast_data[route_code] = {}
43     if date not in forecast_data[route_code]:
44         forecast_data[route_code][date] = {}
45     forecast_data[route_code][date][minute_start] = volume
46
47 # 生成运输需求（实际应用中应根据预测结果生成）
48 transport_demands = []
49 # 遍历预测数据以生成运输需求
50 for route_code in forecast_data:
51     for date in forecast_data[route_code]:
52         # 查找匹配的路径信息
53         matching_routes = [r for r in routes if r['code'] == route_code]
54         if matching_routes:
55             route = matching_routes[0]
56             # 计算总包裹量
57             total_volume = sum(forecast_data[route_code][date].values())
58             # 确保 dispatch_time 是字符串格式
59             if isinstance(route['dispatch_time'], datetime):
60                 dispatch_time_str = route['dispatch_time'].strftime('%H:%M:%S')
61             else:
62                 dispatch_time_str = route['dispatch_time'].isoformat(timespec='seconds')
63                 latest_dispatch_time = datetime.strptime(dispatch_time_str, format='%H:%M:%S')
64             transport_demands.append({
65                 'route': route,
66                 'volume': total_volume,
67                 'latest_dispatch_time': latest_dispatch_time
68             })
69
70 # 确保所有流量都是有效的数值
71 for demand in transport_demands:
72     # 将非数值转换为NaN，然后检查是否为NaN
73     demand['volume'] = pd.to_numeric(demand['volume'], errors='coerce')
74     if pd.isna(demand['volume']):
75         demand['volume'] = 0 # 或者设置为一个合理的默认值
76
77 own_vehicles = 5 # 自有车辆数
78 vehicle_capacity = 1000 # 车辆满载包裹量
79 loading_unloading_time = 45 # 装车及卸车时长（分钟）
80
81 # 权重系数（根据优先级设定）
82 W1 = 1 # 自有车辆周转率权重
83 W2 = 1 # 车辆均包裹量权重
84 W3 = 10 # 总成本权重
85
86 # 定义多目标优化问题
87 prob = pulp.LpProblem(name="Vehicle_Scheduling_Optimization", pulp.LpMinimize)
88
89 # 决策变量
90 x = {} # x[j][k]: 需求j由自有车辆k承运时为1，否则为0
91 y = {} # y[j]: 需求j由外部车承运时为1，否则为0
92 t = {} # t[j]: 需求j的实际发运时间（分钟数）
93
94 # 确保有运输需求
95 if not transport_demands:
96     print("没有运输需求数据。")
97     exit()
98
99 for j, demand in enumerate(transport_demands):
100     x[j] = {}
101     for k in range(own_vehicles):
102         x[j][k] = pulp.LpVariable(name=f"x_{j}_{k}", lowBound=0, upBound=1, pulp.LpInteger)
103         y[j] = pulp.LpVariable(name=f"y_{j}", lowBound=0, upBound=1, pulp.LpInteger)
104         t[j] = pulp.LpVariable(name=f"t_{j}", lowBound=0, upBound=1440, pulp.LpInteger) # 0-1440分钟 (00:00-24:00)
105
106 # 目标函数组件
107 # 1. 自有车辆周转率
108 turnover_rate = pulp.lpSum([pulp.lpSum([x[j][k] for k in range(own_vehicles)]) for j in range(len(transport_demands))])
109 # 2. 车辆均包裹量
110 vehicle_avg_parcel = pulp.lpSum([demand['volume'] for demand in transport_demands])
111 # 3. 总成本
112 total_cost = own_vehicles * 100 + pulp.lpSum([x[j][k] * turnover_rate + demand['volume'] * vehicle_avg_parcel + demand['volume'] * vehicle_capacity * y[j] for j in range(len(transport_demands)) for k in range(own_vehicles)]) + pulp.lpSum([y[j] * transport_demands[j]['route']['external_cost'] for j in range(len(transport_demands))])
113
114 # 综合目标函数
115 prob += W1 * turnover_rate + W2 * vehicle_avg_parcel + W3 * total_cost
116
117 # 约束条件
118 # 1. 需求覆盖
119 for j in range(len(transport_demands)):
120     prob += pulp.lpSum([x[j][k] for k in range(own_vehicles)]) + y[j] == 1
121
122 # 2. 发运时间限制
123 for j in range(len(transport_demands)):
124     latest_dispatch_time = transport_demands[j]['latest_dispatch_time'].hour * 60 + transport_demands[j]['latest_dispatch_time'].minute
125     prob += t[j] <= latest_dispatch_time
126
127 # 3. 车辆数量限制
128 for k in range(own_vehicles):
129     prob += pulp.lpSum([x[j][k] for j in range(len(transport_demands))]) <= own_vehicles
130
131 # 4. 流量满载约束
132 for j in range(len(transport_demands)):
133     prob += pulp.lpSum([x[j][k] for k in range(own_vehicles)]) * vehicle_capacity >= transport_demands[j]['volume']
134     1 - pulp.lpSum([x[j][k] for k in range(own_vehicles)]) * vehicle_capacity
135
136 # 求解问题
137 prob.solve()
138
139 # 准备写入Excel的结果
140 result = []
141
142 for j in range(len(transport_demands)):
143     route_code = transport_demands[j]['route']['code']
144     date = '2024/12/16'
145     vehicle = None
146     dispatch_time = None
147
148     # 确定发运车辆类型和发运时间
149     for k in range(own_vehicles):
150         if pulp.value(x[j][k]) == 1:
151             vehicle = f"自有车 {k+1}"
152             dispatch_time = pulp.value(t[j])
153             break
154     if vehicle is None and pulp.value(y[j]) == 1:
155         vehicle = "外部车"
156         dispatch_time = pulp.value(t[j])
157
158     # 转换发运时间为字符串格式
159     if dispatch_time is not None:
160         dispatch_time_str = (datetime.strptime(date, format='%Y/%m/%d') + timedelta(minutes=dispatch_time)).strftime('%Y/%m/%d %H:%M:%S')
161     else:
162         dispatch_time_str = "未安排"
163
164     # 添加到结果列表
165     result.append({
166         '线路编码': route_code,
167         '日期': date,
168         '预计发运时间': dispatch_time_str,
169         '发运车辆': vehicle
170     })
171
172 df_result = pd.DataFrame(result)
173 df_result.to_excel(excel_writer='D:\调度结果.xlsx', index=False)
174

```

```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import numpy as np
4
5 # 设置全局字体为支持中文字体
6 plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体
7 plt.rcParams['axes.unicode_minus'] = False # 正确显示负号
8
9 file_path = 'D:/结果表4.xlsx'
10 df = pd.read_excel(file_path, header=0, names=['线路编码', '日期', '预计发运时间', '是否使用容器', '发运车辆'])
11
12 df['完整时间'] = df['日期'].astype(str) + ' ' + df['预计发运时间'].astype(str)
13 df['完整时间'] = pd.to_datetime(df['完整时间'], format='%Y/%m/%d %H:%M:%S')
14
15 # 创建一个时间序列, 以小时为单位递增
16 start_time = df['完整时间'].min()
17 end_time = df['完整时间'].max()
18 time_range = pd.date_range(start=start_time, end=end_time, freq='H') # 修改这里的频率参数为1小时间隔
19
20 # 初始化一个矩阵来存储自有车辆和外部车辆的次数
21 heatmap_data = np.zeros((len(time_range), 2)) # 两列分别代表自有车辆和外部车辆
22
23 # 统计每个时间点的自有车辆和外部车辆的出现次数
24 for i, time in enumerate(time_range):
25     # 统计自有车辆的次数 (根据第五列的前两个字判断)
26     own_vehicle_count = df[(df['完整时间'].dt.floor('H') == time.floor('H')) &
27                             (df['发运车辆'].str[:2] == '自有')].shape[0]
28     # 统计外部车辆的次数 (根据第五列的前两个字判断)
29     external_vehicle_count = df[(df['完整时间'].dt.floor('H') == time.floor('H')) &
30                                 (df['发运车辆'].str[:2] != '自有')].shape[0]
31     heatmap_data[i] = [own_vehicle_count, external_vehicle_count]
32
33 heatmap_data = heatmap_data.astype(int)
34
35 # 绘制车次时间变化折线图
36 plt.figure(figsize=(15, 10))
37 plt.plot(*args: time_range, heatmap_data[:, 0], label='使用容器后自有车辆', marker='o')
38 plt.plot(*args: time_range, heatmap_data[:, 1], label='使用容器后外部车辆', marker='s')
39
40 # 设置图表标题和坐标轴标签
41 plt.title('车次时间变化折线图')
42 plt.xlabel('时间')
43 plt.ylabel('车次')
44 plt.legend()
45 plt.grid(True)
46 plt.xticks(rotation=45)
47 plt.tight_layout()
48 plt.show()

```

```

1  import pandas as pd
2
3  file_path = 'D:附件2.xlsx'
4  df = pd.read_excel(file_path)
5
6  # 确保日期列是 datetime 类型
7  df['日期'] = pd.to_datetime(df['日期'])
8
9  # 筛选从 2024/11/30 21:00:00 到 2024/12/15 05:50:00 的数据
10 start_date = pd.Timestamp('2024-11-30 21:00:00')
11 end_date = pd.Timestamp('2024-12-15 05:50:00')
12 filtered_df = df[(df['日期'] >= start_date) & (df['日期'] <= end_date)]
13
14 # 创建一个新的日期列，将 21:00:00 到 23:59:59 的数据归为第二天
15 filtered_df['统计日期'] = filtered_df['日期'].apply(lambda x: x + pd.Timedelta(days=1) if x.hour >= 21 else x)
16 filtered_df['统计日期'] = filtered_df['统计日期'].dt.date
17
18 # 按线路编码和统计日期分组，并计算包裹量总和
19 result = filtered_df.groupby(['线路编码', '统计日期'])['包裹量'].sum().reset_index()
20
21 print(result)
22
23 result.to_excel('D:每天包裹量统计.xlsx', index=False)

```



```

22 import pandas as pd
23
24 df = pd.read_excel('D:aaaa.xlsx')
25
26 df['车队'] = df['发运车辆'].str.extract(r'自有车辆_车队(\d+)')[0]
27 df['车队'] = df['车队'].fillna('其他') # 非自有车辆
28
29 # 按车队分组并计算总包裹量
30 team_total_volume = df.groupby('车队')['总运货量'].sum()
31
32 # 定义自有车辆数量的字典
33 team_vehicle_count = {
34     '1': 29,
35     '2': 14,
36     '3': 36,
37     '4': 16,
38     '5': 15
39 }
40
41 # 将 Series 转换为字典
42 team_total_volume_dict = team_total_volume.to_dict()
43
44 # 计算每个车队的总包裹量除以对应的自有车辆数量
45 team_total_volume_divided = {}
46 for team, volume in team_total_volume_dict.items():
47     # 确保车队编号是字符串形式，与字典中的键匹配
48     team_str = str(team)
49     if team_str in team_vehicle_count:
50         team_total_volume_divided[team_str] = volume / team_vehicle_count[team_str]
51     else:
52         team_total_volume_divided[team_str] = volume # 对于其他车队，保持原值
53
54 # 输出结果
55 print("每个车队的总包裹量:")
56 print(team_total_volume)
57 print("\n每个车队的总包裹量除以自有车辆数量:")
58 print(team_total_volume_divided)
59 # import pandas as pd

```



```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import numpy as np
4 from datetime import datetime
5
6 plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体
7 plt.rcParams['axes.unicode_minus'] = False # 正确显示负号
8
9 file_path = 'D:serial_point_scheduling_results_final.xlsx'
10 df = pd.read_excel(file_path, usecols=[1, 2], header=0, names=['预计发运时间', '发运车辆'])
11
12 # 将字符串转换为datetime对象
13 df['预计发运时间'] = pd.to_datetime(df['预计发运时间'])
14
15 # 创建一个时间序列，以小时为单位递增
16 start_time = df['预计发运时间'].min()
17 end_time = df['预计发运时间'].max()
18 time_range = pd.date_range(start=start_time, end=end_time, freq='H')
19
20 # 初始化一个矩阵来存储自有车辆和外部车辆的次数
21 heatmap_data = np.zeros((len(time_range), 2))
22
23 # 统计每个时间点的自有车辆和外部车辆的出现次数
24 for i, time in enumerate(time_range):
25     # 统计自有车辆的次数
26     own_vehicle_count = df[(df['预计发运时间'].dt.floor('H') == time.floor('H')) & # 使用floor方法精确到小时
27                             (df['发运车辆'].str.contains('自有车辆'))].shape[0]
28     # 统计外部车辆的次数
29     external_vehicle_count = df[(df['预计发运时间'].dt.floor('H') == time.floor('H')) & # 使用floor方法精确到小时
30                                 (~df['发运车辆'].str.contains('自有车辆'))].shape[0]
31     heatmap_data[i] = [own_vehicle_count, external_vehicle_count]
32
33 # 打印数据以检查
34 print(heatmap_data)
35
36 # 将热力图数据转换为整数类型
37 heatmap_data = heatmap_data.astype(int)
38
39 # 绘制车次时间变化折线图
40 plt.figure(figsize=(15, 10))
41 plt.plot(*args: time_range, heatmap_data[:, 0], label='自有车辆', marker='o')
42 plt.plot(*args: time_range, heatmap_data[:, 1], label='外部车辆', marker='s')
43
44 # 设置图表标题和坐标轴标签
45 plt.title('车次时间变化折线图')
46 plt.xlabel('时间')
47 plt.ylabel('车次')
48 plt.legend()
49 plt.grid(True)
50 plt.xticks(rotation=0)
51 plt.tight_layout()
52 plt.show()

```

```

1 import pandas as pd
2
3 file_path = 'D:aaaa.xlsx' # 替换为你的文件路径
4 df = pd.read_excel(file_path)
5
6 自有车辆记录 = df[df['发运车辆'].str.startswith('自有车辆_车队')]
7
8 # 提取车队编号并添加到 DataFrame
9 自有车辆记录.loc[:, '车队'] = 自有车辆记录['发运车辆'].str.extract(r'自有车辆_车队(\d+)')[0]
10
11 # 统计每个车队的总需求数量
12 车队需求数量 = 自有车辆记录.groupby('车队').size().reset_index(name='总需求数量')
13
14 车队车辆数 = {
15     '1': 29,
16     '2': 14,
17     '3': 36,
18     '4': 16,
19     '5': 15,
20 }
21
22 车队需求数量['车队自有车辆数'] = 车队需求数量['车队'].map(车队车辆数)
23
24 车队需求数量['自有车周转率'] = 车队需求数量['总需求数量'] / 车队需求数量['车队自有车辆数']
25
26 print(车队需求数量[['车队', '自有车周转率']])

```

```

50 import matplotlib.pyplot as plt
51 import seaborn as sns
52 import pandas as pd
53 import numpy as np
54 from datetime import datetime
55
56 # 设置全局字体为支持中文字体
57 plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体
58 plt.rcParams['axes.unicode_minus'] = False # 正确显示负号
59
60 file_path = 'D:iiii.xlsx'
61 df = pd.read_excel(file_path)
62
63 df = df[['预计发运时间', '发运车辆']]
64
65 # 将字符串转换为datetime对象
66 df['预计发运时间'] = pd.to_datetime(df['预计发运时间'])
67
68 # 创建一个时间序列，以小时为单位递增
69 start_time = df['预计发运时间'].min()
70 end_time = df['预计发运时间'].max()
71 time_range = pd.date_range(start=start_time, end=end_time, freq='H') # 修改这里的频率参数为1小时间隔
72
73 # 初始化一个矩阵来存储自有车辆和外部车辆的次数
74 heatmap_data = np.zeros((len(time_range), 2)) # 两列分别代表自有车辆和外部车辆
75
76 # 统计每个时间点的自有车辆和外部车辆的出现次数
77 for i, time in enumerate(time_range):
78     # 统计自有车辆的次数
79     own_vehicle_count = df[(df['预计发运时间'].dt.floor('H') == time.floor('H')) & # 使用floor方法精确到小时
80                            (df['发运车辆'].str.contains('自有车辆'))].shape[0]
81     # 统计外部车辆的次数
82     external_vehicle_count = df[(df['预计发运时间'].dt.floor('H') == time.floor('H')) & # 使用floor方法精确到小时
83                                 (~df['发运车辆'].str.contains('自有车辆'))].shape[0]
84     heatmap_data[i] = [own_vehicle_count, external_vehicle_count]
85
86 # 打印数据以检查
87 print(heatmap_data)
88
89 # 将热力图数据转换为整数类型
90 heatmap_data = heatmap_data.astype(int)
91
92 # 绘制热力图，使用颜色深浅来反映数值大小
93 plt.figure(figsize=(15, 10))
94 sns.heatmap(heatmap_data, cmap='YlGnBu', xticklabels=['自有车辆', '外部车辆'], yticklabels=time_range.strftime('%Y-%m-%d %H:%M'))
95 plt.xlabel('车辆类型')
96 plt.ylabel('时间')
97 plt.title('车次时间分布热力图')
98 plt.tight_layout()
99 plt.show()

```

```

83 # 设置全局字体为支持中文字体
84 plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体
85 plt.rcParams['axes.unicode_minus'] = False # 正确显示负号
86
87 file_path = 'D:iiii.xlsx'
88 df = pd.read_excel(file_path)
89
90 df = df.rename(columns={'线路': '场地'})
91
92 df['场地'] = df['场地'].str.extract(r'(场地\d+)') # 提取场地编号
93
94 df['自有车辆'] = df['发运车辆'].str.startswith('自有车辆').astype(int)
95 df['外部车辆'] = df['发运车辆'].str.startswith('外部车辆').astype(int)
96
97 # 按场地分组并求和
98 grouped = df.groupby('场地')[['自有车辆', '外部车辆']].sum()
99
100 # 计算总车辆数和自有车辆比例
101 grouped['总车辆数'] = grouped.sum(axis=1)
102 grouped['自有车辆比例'] = grouped['自有车辆'] / grouped['总车辆数']
103
104 # 打印每个场地的自有车辆数和外部车辆数
105 print("各场地车辆统计信息:")
106 for venue in grouped.index:
107     own = grouped.loc[venue, '自有车辆']
108     external = grouped.loc[venue, '外部车辆']
109     total = grouped.loc[venue, '总车辆数']
110     ratio = grouped.loc[venue, '自有车辆比例']
111     print(f"场地: {venue}, 自有车辆数: {own}, 外部车辆数: {external}, 总车辆数: {total}, 自有车辆比例: {ratio:.2f}")
112
113 # 准备绘图数据
114 venues = grouped.index
115 own_vehicles = grouped['自有车辆']
116 external_vehicles = grouped['外部车辆']
117 total_vehicles = grouped['总车辆数']
118 own_vehicle_ratios = grouped['自有车辆比例']
119
120 # 绘制气泡图
121 plt.figure(figsize=(8, 6)) # 进一步减小图形尺寸
122 scatter = plt.scatter(external_vehicles, own_vehicles, s=total_vehicles * 30, c=own_vehicle_ratios, cmap='YlGnBu')
123
124 # 添加对角线
125 max_limit = max(max(external_vehicles), max(own_vehicles)) * 1.1
126 plt.plot(*args=[0, max_limit], [0, max_limit], linestyle='--', color='gray', alpha=0.7)
127
128 # 添加颜色条
129 cbar = plt.colorbar(scatter)
130 cbar.set_label('自有车辆比例')
131
132 # 设置图表标题和坐标轴标签
133 plt.title('各场地车辆分配关系气泡图')
134 plt.xlabel('外部车辆数')
135 plt.ylabel('自有车辆数')
136
137 # 添加场地名称标签
138 for i, venue in enumerate(venues):
139     plt.annotate(venue, xy=(external_vehicles[i], own_vehicles[i]), fontsize=7, ha='center')
140
141 # 添加网格
142 plt.grid(visible=True, linestyle='--', alpha=0.6)
143
144 # 设置坐标轴范围
145 plt.xlim(*args=[0, max_limit])
146 plt.ylim(*args=[0, max_limit])
147
148 # 添加注释
149 plt.text(x=0.5, y=-0.15, s='注: 气泡大小表示总车辆数, 颜色深浅表示自有车辆比例',
150         ha='center', fontsize=7, transform=plt.gca().transAxes)
151
152 plt.tight_layout()
153 plt.show()

```

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import matplotlib as mpl
5 from scipy.stats import norm
6 from scipy.interpolate import UnivariateSpline
7
8 # 设置中文字体
9 mpl.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体
10 mpl.rcParams['axes.unicode_minus'] = False # 正确显示负号
11
12 # 读取 Excel 文件
13 file_path = 'D:包裹量比值结果&k.xlsx' # 替换为你的文件路径
14 df = pd.read_excel(file_path)
15
16 # 提取包裹量_attachment3 列的数据
17 data = df['包裹量_attachment3']
18
19 # 将数据归一化到 [0, 1] 范围内
20 normalized_data = (data - np.min(data)) / (np.max(data) - np.min(data))
21
22 # 设置直方图的区间
23 plt.figure(figsize=(12, 6))
24 plt.hist(normalized_data, bins=20, color='skyblue', edgecolor='black', alpha=0.7, density=True, label='归一化直方图')
25
26 # 计算每个区间的中点
27 counts, bin_edges = np.histogram(normalized_data, bins=20, density=True)
28 bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
29
30 # 使用UnivariateSpline生成归一化平滑曲线
31 spline = UnivariateSpline(bin_centers, counts, s=0.5) # s控制平滑程度
32 x_smooth = np.linspace(bin_edges[0], bin_edges[-1], num=1000)
33 y_smooth = spline(x_smooth)
34
35 # 绘制平滑曲线
36 plt.plot(*args: x_smooth, y_smooth, color='mediumblue', linewidth=2, linestyle='--', label='平滑曲线')
37 |
38 plt.title('包裹量的归一化分布直方图')
39 plt.xlabel('归一化包裹量 (0到1)')
40 plt.ylabel('概率密度')
41
42 # 设置 x 轴刻度范围，左右增加空白区域
43 plt.xlim(*args: -0.1, 1.1) # 左右各留出10%的空白区域
44
45 # 添加图例和网格
46 plt.legend(fontsize=12)
47 plt.grid(visible=True, linestyle='--', alpha=0.7)
48
49 plt.tight_layout()
50 plt.show()

```



```

36 # import pandas as pd
37 # import matplotlib.pyplot as plt
38 # from matplotlib.font_manager import FontProperties
39 # import warnings
40 # import numpy as np
41 #
42 # # 忽略警告
43 # warnings.filterwarnings("ignore", category=UserWarning, module="tkinter")
44 #
45 # # 设置中文字体
46 # plt.rcParams['font.sans-serif'] = ['Microsoft YaHei'] # 替换为可用的中文字体
47 # plt.rcParams['axes.unicode_minus'] = False # 用于正常显示负号
48 #
49 # # 读取 Excel 文件
50 # file_path = 'D:1111.xlsx'
51 # df = pd.read_excel(file_path)
52 #
53 # # 数据预处理
54 # df['预计发运时间'] = pd.to_datetime(df['预计发运时间'])
55 # df['日期'] = pd.to_datetime(df['日期'])
56 #
57 # # 提取需要的数据列
58 # dates = df['日期'].dt.date
59 # times = df['预计发运时间'].dt.time
60 # shipping_types = df['发运车辆']
61 # cargo_volumes = df['总运货量']
62 #
63 # # 创建一个时间序列索引
64 # df['datetime'] = pd.to_datetime(df['日期'].astype(str) + ' ' + df['预计发运时间'].dt.time.astype(str))
65 #
66 # # 按日期和时间聚合数据
67 # daily_data = df.groupby([df['datetime'].dt.date, '发运车辆'])['总运货量'].sum().unstack(fill_value=0)
68 #
69 # # 绘制折线图展示每日运货量趋势
70 # plt.figure(figsize=(12, 8))
71 # for vehicle_type in daily_data.columns:
72 #     plt.plot(daily_data.index, daily_data[vehicle_type], marker='o', label=vehicle_type)
73 #
74 # plt.title('每日运货量趋势')
75 # plt.xlabel('日期')
76 # plt.ylabel('运货量')
77 # plt.legend()
78 # plt.grid(True)
79 # plt.xticks(rotation=45)
80 # plt.tight_layout()
81 # plt.show()
82 #
83 # # 绘制柱状图展示车辆类型分布
84 # external_vehicle_count = (shipping_types == '外部车辆').sum()
85 # own_vehicle_count = shipping_types.str.startswith('自有车辆').sum()
86 #
87 # data = [external_vehicle_count, own_vehicle_count]
88 # labels = ['外部车辆', '自有车辆']
89 #
90 # fig, ax = plt.subplots(figsize=(8, 8))
91 # ax.pie(data, labels=labels, autopct='%1.1f%%', startangle=90, explode=(0.1, 0), shadow=True)
92 # ax.set_title('发运车辆类型比例')
93 # plt.axis('equal')
94 # plt.tight_layout() # 确保布局紧凑
95 # plt.show()
96 #
97 # # 绘制运货量随时间变化的散点图
98 # plt.figure(figsize=(12, 6))
99 # for vehicle_type in ['外部车辆', '自有车辆']:
100 #     subset = df[df['发运车辆'] == vehicle_type]
101 #     plt.scatter(subset['datetime'], subset['总运货量'], label=vehicle_type, alpha=0.6)
102 #
103 # plt.title('运货量随时间变化')
104 # plt.xlabel('时间')
105 # plt.ylabel('运货量')
106 # plt.legend()
107 # plt.grid(True)
108 # plt.xticks(rotation=45)
109 # plt.tight_layout()
110 # plt.show()
111 import pandas as pd
112 import matplotlib.pyplot as plt
113 from matplotlib.font_manager import FontProperties
114 import warnings
115 import numpy as np
116
117 # 忽略警告
118 warnings.filterwarnings("ignore", category=UserWarning, module="tkinter")
119
120 # 设置中文字体
121 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei'] # 替换为可用中文字体
122 plt.rcParams['axes.unicode_minus'] = False # 用于正常显示负号
123
124 # 读取 Excel 文件
125 file_path = 'D:1111.xlsx'
126 df = pd.read_excel(file_path)
127
128 # 数据预处理
129 df['预计发运时间'] = pd.to_datetime(df['预计发运时间'])
130 df['日期'] = pd.to_datetime(df['日期'])
131
132 # 提取需要的数据列
133 dates = df['日期'].dt.date
134 times = df['预计发运时间'].dt.time
135 shipping_types = df['发运车辆']
136 cargo_volumes = df['总运货量']
137
138 # 创建一个时间序列索引
139 df['datetime'] = pd.to_datetime(df['日期'].astype(str) + ' ' + df['预计发运时间'].dt.time.astype(str))
140
141 # 按日期和时间聚合数据
142 daily_data = df.groupby([df['datetime'].dt.date, '发运车辆'])['总运货量'].sum().unstack(fill_value=0)
143
144 # 绘制折线图展示每日运货量趋势
145 plt.figure(figsize=(12, 8))
146 for vehicle_type in daily_data.columns:
147     plt.plot(daily_data.index, daily_data[vehicle_type], marker='o', label=vehicle_type)
148
149 plt.title('每日运货量趋势')
150 plt.xlabel('日期')
151 plt.ylabel('运货量')
152 plt.legend()
153 plt.grid(True)
154 plt.xticks(rotation=45)
155 plt.tight_layout()
156 plt.show()

```